URPlanner: A Universal Paradigm for Collision-Free Robotic Motion Planning Based on Deep Reinforcement Learning

Fengkang Ying, *Graduate Student Member, IEEE*, Hanwen Zhang, Haozhe Wang, *Graduate Student Member, IEEE*, Huishi Huang, *Graduate Student Member, IEEE*, and Marcelo H. Ang Jr., *Senior Member, IEEE*

***Abstract*—Collision-free motion planning for redundant robot manipulators in complex environments is yet to be explored. Although recent advancements at the intersection of deep reinforcement learning (DRL) and robotics have highlighted its potential to handle versatile robotic tasks, current DRL-based collision-free motion planners for manipulators are highly costly, hindering their deployment and application. This is due to an overreliance on the minimum distance between the manipulator and obstacles, inadequate exploration and decision making by DRL, and inefficient data acquisition and utilization. In this article, we propose URPlanner, a universal paradigm for collision-free robotic motion planning based on DRL. URPlanner offers several advantages over existing approaches: it is platform agnostic, cost-effective in both training and deployment, and applicable to arbitrary manipulators without solving inverse kinematics. To achieve this, we first develop a parameterized task space and a universal obstacle avoidance reward that is independent of minimum distance. Second, we introduce an augmented policy exploration and evaluation algorithm that can be applied to various DRL algorithms to enhance their performance. Third, we propose an expert data diffusion strategy for efficient policy learning, which can produce a large-scale trajectory dataset from only a few expert demonstrations. Finally, the superiority of the proposed methods is comprehensively verified through experiments.**

***Index Terms*—Augmented policy exploration and evaluation (APE2), collision-free motion planning, expert data diffusion (ED2), parameterized task space, universal obstacle avoidance reward (UOAR).**

Received 14 May 2025; accepted 5 August 2025. Date of publication 20 August 2025; date of current version 8 September 2025. This work was supported by the National Research Foundation, Prime Minister's Office, Singapore, under its Campus for Research Excellence and Technological Enterprise program. The Mens, Manus, and Machina is an interdisciplinary research group of the Singapore MIT Alliance for Research and Technology Centre. This article was recommended for publication by Associate Editor K. A. Hamed and Editor J. Kober upon evaluation of the reviewers' comments. *(Corresponding author: Haozhe Wang.)*

Fengkang Ying and Haozhe Wang are with the Integrative Sciences and Engineering Programme, NUS Graduate School, National University of Singapore, Singapore 119077, and also with the Advanced Robotics Centre, National University of Singapore, Singapore 117608 (e-mail: fengkang@u.nus.edu; wang_haozhe@u.nus.edu).

Hanwen Zhang, Huishi Huang, and Marcelo H. Ang Jr. are with the Department of Mechanical Engineering, College of Design and Engineering, National University of Singapore, Singapore 117575, and also with the Advanced Robotics Centre, National University of Singapore, Singapore 117608 (e-mail: hanwen.zhang@u.nus.edu; huishi.huang@u.nus.edu; mpeangh@nus.edu.sg).

This article has supplementary downloadable material available at https://doi.org/10.1109/TRO.2025.3600138, provided by the authors.

Digital Object Identifier 10.1109/TRO.2025.3600138

I. Introduction

A. Motivation and Related Work

In RECENT years, robot manipulators have been increasingly utilized across a diverse array of environments and scenarios, ranging from industrial manufacturing and processing to daily service and recreation [1]. Particularly, redundant manipulators can move more flexibly and are more adaptable to the growing complexity of the environment they operate in due to their enhanced ability to avoid collision and singularity [2]. However, the increased environmental complexity and robotic redundancy have also significantly complicated the collision-free motion planning problem.

Traditional motion planning (TMP) methods, such as rapidly exploring random trees (RRT) and probabilistic roadmap (PRM), are probabilistically complete [3]. This property ensures that a solution can always be found if it exists, as long as the search time approaches infinite. However, the path they find is nonoptimal [4]. Their improved variants RRT* and PRM* can find asymptotically optimal paths, but the optimization ability is limited as they do not leverage information from past experiences [5]. In comparison, learning-based planning methods can learn from previously obtained samples to improve planning and optimization efficiency.

Besides the optimization problem, these TMP methods also exhibit several deficiencies in other aspects. For example, they often fall short of efficiently handling the high-dimensional space, causing the degradation of performance as the complexity of the environment increases [6]. However, learning-based methods can perceive and represent high-dimensional environmental states by neural networks, making the planning more tractable in complex environments. Moreover, for TMP methods, each planning task requires repeatedly searching the whole environment from scratch. In contrast, learning-based methods can be trained to directly infer an optimal solution without going through the search process.

In addition, for the motion planning of manipulators, these traditional methods rely on solving inverse kinematics (IK) to map the trajectories from Cartesian space to joint space. In this process, extra considerations regarding the collision avoidance of the entire manipulator are required [7]. For manipulators with less than six degrees of freedom (DoFs), TMP methods usually go through all possible IK configurations to find the optimal one. However, the aforementioned procedures can be quite complicated and time consuming for redundant manipulators since

1941-0468 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

there are infinite IK solutions for each waypoint of the trajectory. By contrast, some researchers have designed learning-based methods that can bypass solving the IK and directly learn a policy capable of reaching the target point by constantly controlling the joint rotation of manipulators. Such methods exhibit better universality as they can be applied to manipulators with arbitrary DoFs without designing a customized IK solving and planning method [7], [8], [9].

Recently, the intersection of deep reinforcement learning (DRL) and robotics has piqued much attention since it enables robots to autonomously learn optimal policies for versatile tasks. Such policies can adapt to diverse and unpredictable environments and can continuously improve performance through self-learning and adaptation [10], [11]. Thus, DRL is particularly suited for addressing the intricate robotic motion planning problem that is extremely challenging for traditional methods. For example, regarding trajectory generation tasks, Ying et al. [8] proposed an extensively explored and evaluated actor–critic (E3AC) algorithm to realize an IK-free motion planning scheme, meaning that the method is independent of solving the IK and can be applied to manipulators with any DoFs. Ying et al. [9] proposed a multipolicy agent that could effectively handle sequential trajectory planning for multiprocess robotic tasks. In terms of motion planning for pick-and-place tasks, Hu et al. [12] leveraged the proximal policy optimization algorithm and inverse reinforcement learning (RL) techniques to plan the motion for grasping living objects such as fish. The deep Q-network algorithm has been widely used to plan the motion sequence of grasping cluttered objects [13], [14], [15]. For compliant motion planning problem, Hou et al. [16], [17] has adopted deep deterministic policy gradient (DDPG), twin delayed DDPG (TD3), and soft actor–critic (SAC) algorithms to implement multiple peg-in-hole assembly. Xu et al. [18] applied the SAC algorithm to enable a robot hand to learn the techniques of playing the piano. For soft robot arms and deformable manipulation, DRL has been leveraged to train a pneumatic-actuated soft robot to throw objects [19] and to manipulate deformable ropes [20].

Despite the advancements in DRL-based motion planning for manipulators, three main deficiencies persistently hinder its effectiveness and efficiency in real-world deployment.

First, the obstacle avoidance ability of the manipulators is rarely considered in the aforementioned studies and remains relatively unexplored [11]. Regarding this, the design of appropriate and task-specific reward functions is crucial, as it directly affects the learning outcomes of DRL policies [3]. One approach is to set the reward as zero when a collision is detected [5], [21]. However, such sparse rewards result in many different samples with the same zero feedback, significantly degrading the policy learning process [7]. As for dense rewards, current methods for obstacle avoidance heavily rely on the minimum distance (MD) information between the robot and obstacles. For example, Sui et al. [22] designed a linear reward that is proportional to the MD. In comparison, Ying et al. [7] proposed a quadratic reward function based on the MD, which can better balance obstacle avoidance against action exploration. However, these methods idealistically assume that the MD can be accurately and efficiently obtained, while, in practice, it requires the geometry of all the objects, sufficient sensors, and complex analyses.

Second, current DRL methods exhibit limitations in their exploration and decision-making capabilities, leading to inefficient and unstable training performance [7]. Regarding the exploration, most DRL algorithms, such as DDPG and TD3, produce only a single action based on the current state and then explore only once by adding a random noise. However, the single-action approach in exploration causes a lack of diversity and scalability, and the randomness may even worsen the performance of the original action. To enhance exploration, previous studies have introduced a multiactor structure to generate multiple action candidates [7], [23]. However, it also made the process more inefficient. Similarly, regarding decision-making, there might be prejudice when the values of actions are estimated by only a single critic. To address this issue, an extensive evaluation architecture has been proposed, where multiple critics are integrated to decide the optimal action in a more unbiased way [23], [24]. Nevertheless, the critic networks typically exhibit poor performance during early training stages, leading to unsatisfactory value evaluation.

Third, data acquisition and utilization pose significant challenges to deploying DRL algorithms. While DRL does not necessitate labeled datasets, policy optimization still relies on large-scale data obtained by random interaction without specific directions. Thus, it is difficult for DRL to optimize the current policy efficiently unless ample high-quality data have been obtained [7]. The phenomenon is more severe in the early training stage, as the networks have not been sufficiently trained. Regarding the problem, prioritized experience replay has been leveraged to make full use of the data with relatively higher quality [25], [26]. However, as a technique for DRL with sparse reward, its effect can be limited if the reward is dense [8]. Ying et al. [9] improved the learning efficiency by sampling data with high reward in priority, but ranking large-scale data greatly increases the computational costs. Previous studies have also used expert demonstrations to provide high-quality data, which are beneficial to the training of DRL [8], [25]. Nevertheless, acquiring substantial expert data requires extensive manual effort, while a small number of demonstrations may have a very limited effect on training DRL policies.

B. Contributions and Organizations of This Article

In response to the aforementioned discussions, this article proposes a universal paradigm for collision-free robotic motion planning based on DRL, named URPlanner. URPlanner has several intriguing advantages over existing DRL-based methods. First, we use parameterized space to represent the environment, which makes URPlanner platform agnostic and highly cost-effective in both training and deployment. Second, the trained policy can be directly applied to robots in simulators or the real world without requiring fine-tuning. This is possible because both policy training and trajectory generation use the real robot's kinematic parameters in the parameterized space.

The definition of some critical terms in this article is given as follows.

1) *Platform Agnostic:* In our article, we assume static obstacle layouts, and the term *platform-agnostic* refers specifically to two aspects: a) the training of our motion planning scheme is independent of interaction with any specific simulated or real-world robotic platform, due to the proposed parameterized task space; and b) the motion planning policy trained in the parameterized task space can be applied to the corresponding task scenario built on any simulated or real-world platform.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

2) *Universal:* The term *universal* means that our proposed URPlanner can, in theory, be applied to train any manipulator in any given task scenario. However, the Denavit–Hartenberg (DH) parameters of the manipulator and the radii of the cylinders encapsulating the links should be provided.

The main contributions of this article are as follows.

1) A universal obstacle avoidance reward (UOAR) is designed. The reward is dense, independent of MD, and can be easily calculated. To achieve this, we first build a parameterized environment where the obstacles and manipulator are represented by the composition of bounding boxes and line segments, respectively. Then, the overlap of each line segment with each bounding box is inferred as the UOAR. Finally, it is combined with a pose reward to solve the general robotic motion planning problem based on DRL.
2) An augmented policy exploration and evaluation (APE2) algorithm is proposed. APE2 can be applied to various DRL algorithms, reinforcing their exploration and decision-making capabilities. It generates diverse action candidates under every state by enhanced action exploration. Then, a hybrid policy evaluation approach considering both immediate and long-term environmental feedback is proposed, which provides a more accurate and comprehensive evaluation throughout the training process.
3) An expert data diffusion (ED2) strategy is proposed, where abundant expert data are autonomously generated to facilitate the policy learning of DRL. First, a diffusion model is exploited to produce a large-scale trajectory dataset from only a few demonstrations, minimizing the need for manual participation. Then, two different methods for integrating the dataset with DRL are explored and compared: one creates a synergy between behavior cloning (BC) and DRL, while the other directly utilizes the dataset to supply additional high-quality samples for policy optimization.

The rest of this article is organized as follows. Section II formulates the motion planning problem. Section III describes the details of environment parameterization and the proposed UOAR. Section IV introduces the proposed APE2 algorithm. Section V presents the design of ED2. Section VI summarizes the overall implementation of URPlanner. In Section VII, various experiments are conducted and discussed. Section VIII presents further discussions on the universality of URPlanner, its generalization across different scenarios, the strategies for providing demonstrations to ED2, and some extensions of URPlanner. Finally, Section IX concludes this article.

II. Problem Formulation

In this section, we present a general definition of collision-free motion planning problems based on DRL.

For manipulators, a motion planning problem is to find a sequence of collision-free robot configurations represented as waypoints and the corresponding joint angles [27]. The problem can be formulated as a Markov decision process. At time step t, the agent captures the state $\boldsymbol{s}_t$ (such as the target pose) of the environment and takes an action $\boldsymbol{a}_t$ (such as the joint rotations). The interaction transits the state to $\boldsymbol{s}_{t+1}$, and a reward r_t is obtained as the environmental feedback.

By defining the motion planning of manipulators as a Markov decision process, DRL algorithms can be employed to learn an optimal policy π^* by maximizing the future cumulative reward G_t

$$\begin{aligned} G_t &= r_t + \xi r_{t+1} + \xi^2 r_{t+2} + \cdots + \xi^{n-t} r_n \\ &= r_t + \xi G_{t+1} \end{aligned} \tag{1}$$

where ξ is a discount factor. G_t, also known as the long-term return, represents the cumulative reward an agent expects to receive over time, starting from the current state and following a given policy. In DRL, the Q-value produced by the critic network serves as an estimate of the long-term return.

In comparison to [7], we remove the MD information from $\boldsymbol{s}_t$ to make it more general and practical

$$\boldsymbol{s}_t = [\boldsymbol{q}_t, \boldsymbol{p}_{\mathrm{T}}, \boldsymbol{p}_{\mathrm{G}}, \Delta \mathrm{p}, \Delta \mathrm{o}, D]\,. \tag{2}$$

$\boldsymbol{q}_t$ is the configuration (i.e., joint angles) of the manipulator. $\boldsymbol{p}_{\mathrm{T}}$ and $\boldsymbol{p}_{\mathrm{G}}$ are the pose of the manipulator's tool center point and the goal, respectively. $\Delta \mathrm{p} = [\Delta x, \Delta y, \Delta z]$ and $\Delta \mathrm{o} = [\Delta \alpha, \Delta \beta, \Delta \gamma]$ are the position and orientation deviation between the tool center point and the goal, respectively. D is a Boolean variable that equals 1 when the pose of tool center point is within the allowable errors.

Unlike previous studies that learn the translation and rotation of a manipulator's tool center point for motion planning [16], [17], [28], we let the manipulator directly learn sequential rotations of its joints to bypass IK. Hence, $\boldsymbol{a}_t$ is defined as

$$\boldsymbol{a}_t = [\Delta q_1, \Delta q_2, \ldots, \Delta q_i] \tag{3}$$

where Δq_i is the angular increment of joint i.

III. Space Parameterization and Reward Shaping

In this section, the task space for robotic motion planning is parameterized, with obstacles and robot manipulators described by mathematical models. A UOAR independent of MD information is designed and combined with pose rewards based on the parameterized space.

A. Task Space Parameterization

As shown in Fig. 1, the spatial occupancy of an obstacle with arbitrary geometry can be approximated using one or more axis-aligned bounding boxes (AABBs) [26]. For irregularly shaped obstacles, we recommend using multiple boxes to better approximate their spatial occupancy and minimize unnecessary reserved space. The space enclosed by AABB can be expressed by a point set defined as $Box = \{(x, y, z) \mid x \in [x_{\min}, x_{\max}], y \in [y_{\min}, y_{\max}], z \in [z_{\min}, z_{\max}]\}$.

For a given manipulator, define O_i as the coordinate origin of joint i. The geometric relationship between O_i and O_{i-1} can be represented by DH parameters: link length a_i, link twist α_i, link offset d_i, and joint angle q_i. Based on the DH parameters, the homogeneous transformation matrix between two adjacent joints is defined as

$$T_i^{i-1} = \mathrm{Rot}(\alpha_i)\mathrm{Trans}(a_i)\mathrm{Rot}(q_i)\mathrm{Trans}(d_i). \tag{4}$$

Furthermore, defining the robot base coordinate as the world coordinate system, the pose of joint i can be computed by

$$T_i^0 = T_1^0 T_2^1 \cdots T_i^{i-1} = \begin{bmatrix} \boldsymbol{R}_i^0 & \boldsymbol{P}_i^0 \\ \boldsymbol{0} & 1 \end{bmatrix}_{4\times 4} \tag{5}$$

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

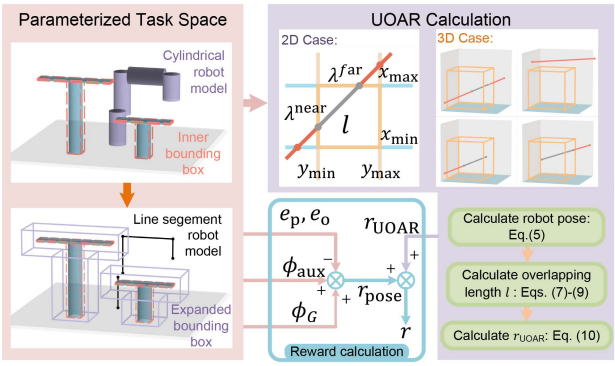


Fig. 1. Schematic diagram of the parameterized space and reward calculation. The manipulator is represented as line segments encapsulated by cylinders, while the obstacles are modeled as expanded bounding boxes, incorporating the cylinder's radius along with a safety offset. UOAR is designed based on the overlap length between the line segments and bounding boxes. This reward encourages the DRL agent to learn collision-free motion planning by minimizing the overlap length, thereby ensuring safe and efficient trajectories.

where $\boldsymbol{R}_i^0$ and $\boldsymbol{P}_i^0 = [x_i^0, y_i^0, z_i^0]^{\mathrm{T}}$ describe the orientation and position of joint i in the world coordinate system, respectively.

Defining $\boldsymbol{p}_i(x_i^0, y_i^0, z_i^0)$ as the coordinates of joint i in the world coordinate system, each link of the manipulator can be simplified and parameterized as a line segment $\boldsymbol{s}_i$ enclosed by a cylinder with a radius a_{r}, denoted as

$$\boldsymbol{s}_i^{i-1} = \boldsymbol{p}_{i-1} + \overrightarrow{\boldsymbol{p}_{i-1}\boldsymbol{p}_i} \cdot \lambda, \qquad 0 \le \lambda \le 1. \tag{6}$$

The parameterized model of any given manipulator can be easily obtained according to its DH parameters. The parameter can be manually modified if necessary so that the links can be evenly encapsulated by cylinders. In addition, some adjacent link pairs (such as links 1 and 2, links 3 and 4, links 5 and 6, and links 7 and 8 of the KUKA LBR iiwa robot or Franka Emika Panda robot) can be consolidated into a single link.

Finally, as shown in Fig. 1, by expanding the bounding boxes to include the radius a_{r} of the simplified robot model and a safety offset a_{o}, the whole collision-free motion planning problem for manipulators can be transformed into ensuring that the line segments avoid intersections with the expanded bounding boxes during motion. Here, a_{o} establishes a buffer zone between the manipulator and surrounding obstacles. This approach aligns with the principles of clearance-based motion planning, which aims not only to prevent collisions but also to maintain a safe distance from obstacles throughout the motion.

Remark 1: The parameterized task space offers significant advantages for DRL-based motion planning. First, it enables efficient calculation of overlaps between the manipulator and obstacles, facilitating the design of the proposed UOAR, which is independent of the MD. Second, it allows any required information for motion planning to be analytically computed. This feature makes URPlanner platform agnostic and highly cost-effective in both training and deployment compared to existing methods [5], [7], [8], [9], [26].

B. Universal Obstacle Avoidance Reward

Given a parameterized link $\boldsymbol{s}$ defined by (6), its starting point and ending point in the world coordinate system can be determined using the transformation matrix T_i^0 from (5). Let T_0^{Box} denote the homogeneous transformation matrix that maps the world coordinate system to the bounding box's coordinate system. Using T_0^{Box} and T_i^0, the coordinates of the starting and ending points in the bounding box's coordinate system, represented as $\boldsymbol{p}_{\mathrm{s}}(x_{\mathrm{s}}, y_{\mathrm{s}}, z_{\mathrm{s}})$ and $\boldsymbol{p}_{\mathrm{e}}(x_{\mathrm{e}}, y_{\mathrm{e}}, z_{\mathrm{e}})$, respectively, can be obtained by multiplying T_0^{Box} with T_i^0.

For a bounding box with six planes, the intersections of the line (on which the segment lies) with the two planes perpendicular to the x-axis can be represented as $\boldsymbol{p}_{\mathrm{s}} + \overrightarrow{\boldsymbol{p}_{\mathrm{s}}\boldsymbol{p}_{\mathrm{e}}} \cdot \lambda$ with the corresponding values of λ. Representing the two planes as $x = x_{\min}$ and $x = x_{\max}$, λ can be solved by

$$\begin{cases} \lambda_x^{\mathrm{near}} = \min\left(\frac{x_{\min} - x_{\mathrm{s}}}{x_{\mathrm{e}} - x_{\mathrm{s}}}, \frac{x_{\max} - x_{\mathrm{s}}}{x_{\mathrm{e}} - x_{\mathrm{s}}}\right) \\ \lambda_x^{\mathrm{far}} = \max\left(\frac{x_{\min} - x_{\mathrm{s}}}{x_{\mathrm{e}} - x_{\mathrm{s}}}, \frac{x_{\max} - x_{\mathrm{s}}}{x_{\mathrm{e}} - x_{\mathrm{s}}}\right). \end{cases} \tag{7}$$

The intersections with the planes perpendicular to y and z axes can be solved using the same logic, respectively.

If the line segment intersects with the given bounding box, the following condition must be satisfied:

$$\max\left(\lambda_x^{\mathrm{near}}, \lambda_y^{\mathrm{near}}, \lambda_z^{\mathrm{near}}, 0\right) \le \min\left(\lambda_x^{\mathrm{far}}, \lambda_y^{\mathrm{far}}, \lambda_z^{\mathrm{far}}, 1\right). \tag{8}$$

The method described by (7) and (8) has been broadly adopted to detect collisions in previous studies, such as [29]. However, based on the collision information, only a sparse reward function can be designed, such as assigning a reward of zero when a collision is detected [5], [21]. Such sparse rewards often hinder the efficiency of the policy learning process. In cases where no collisions are detected, prior studies like [26] and [30] have proposed calculating the MD between the obstacle and the robot to design a dense reward. This approach provides more informative guidance, but is often computationally expensive. In this work, we demonstrate that the overlap length between line segments and expanded bounding boxes can be leveraged to design a dense reward that effectively guides the learning of obstacle avoidance policies while maintaining computational efficiency.

When the link $\boldsymbol{s}$ intersects with the bounding box, $\lambda^{\mathrm{near}} = \max(\lambda_x^{\mathrm{near}}, \lambda_y^{\mathrm{near}}, \lambda_z^{\mathrm{near}}, 0)$ and $\lambda^{\mathrm{far}} = \min(\lambda_x^{\mathrm{far}}, \lambda_y^{\mathrm{far}}, \lambda_z^{\mathrm{far}}, 1)$. Then, the length of the overlapping part of the link is

$$l = |\lambda^{\mathrm{far}} - \lambda^{\mathrm{near}}| \cdot \left|\overrightarrow{\boldsymbol{p}_{\mathrm{s}}\boldsymbol{p}_{\mathrm{e}}}\right|. \tag{9}$$

For a parameterized task space containing G bounding boxes (representing the obstacles) and J line segments (representing the manipulator), let l_{jg} represent the overlapping length between line segment j and bounding box g, and L_j represent the length of line segment j. Then, the proposed UOAR is defined as

$$r_{\mathrm{UOAR}} = -\frac{\sum_{j=1}^{\mathrm{J}} \sum_{g=1}^{\mathrm{G}} l_{jg}}{\sum_{j=1}^{\mathrm{J}} L_j}. \tag{10}$$

Remark 2: Compared with existing DRL-based collision-free motion planning methods, the proposed UOAR transforms the complex problem of calculating MDs into a simpler problem of avoiding overlaps with the bounding boxes. This makes the training process highly cost-effective and enhances the generalization capability across obstacles with various geometries.

To calculate the pose reward, the tool center point's pose ($\boldsymbol{R}_{\mathrm{T}}$ and $\boldsymbol{P}_{\mathrm{T}}$) can be inferred by (5). $\boldsymbol{R}_{\mathrm{T}}$ is converted to the corresponding Euler angles with respect to the goal, denoted as $(\alpha_{\mathrm{T}}^{\mathrm{G}}, \beta_{\mathrm{T}}^{\mathrm{G}}, \gamma_{\mathrm{T}}^{\mathrm{G}})$. Define $e_{\mathrm{p}} = \left|\overrightarrow{\boldsymbol{p}_{\mathrm{T}}\boldsymbol{p}_{\mathrm{G}}}\right|$ and $e_{\mathrm{o}} = |\alpha_{\mathrm{T}}^{\mathrm{G}}| + |\beta_{\mathrm{T}}^{\mathrm{G}}| + |\gamma_{\mathrm{T}}^{\mathrm{G}}|$ as the position error and orientation error, respectively. Note

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

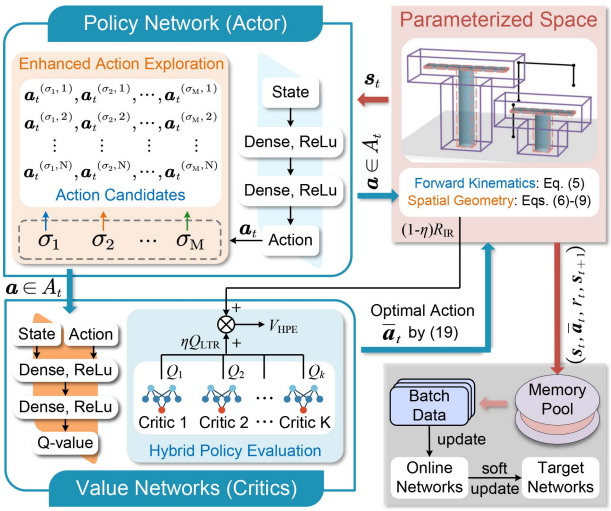


Fig. 2. Framework of the APE2 algorithm. For each state, APE2 efficiently generates a large pool of action candidates through enhanced action exploration. During hybrid policy evaluation, the combination of the average Q-value from multiple critics, Q_{LTR}, and the immediate return, R_{IR}, ensures a more accurate and comprehensive assessment throughout the training process. The agent then executes the action candidate with the highest evaluated value.

that both errors are normalized. Then, the pose reward is defined as

$$r_{\mathrm{pose}} = -\left(e_{\mathrm{p}} + e_{\mathrm{o}}\right) + \phi_{\mathrm{aux}} + \phi_{\mathrm{G}} \tag{11}$$

where ϕ_{G} equals 1 if e_{p} and e_{o} are within allowable ranges, indicating that the tool center point has reached the goal. ϕ_{aux} is an auxiliary reward designed to further reduce the pose error. The setup of ϕ_{aux} follows [9, eqs. (12) and (13).].

Finally, the integrative reward considering both obstacle avoidance and pose control can be defined as

$$r = r_{\mathrm{pose}} + \zeta r_{\mathrm{UOAR}} \tag{12}$$

where ζ is a weight coefficient.

IV. APE2 Algorithm

In this section, the detailed implementation of the proposed APE2 algorithm is introduced.

A. Enhanced Action Exploration

In DRL, a mapping from $\boldsymbol{s}_t$ to $\boldsymbol{a}_t$ is created by the online actor μ with network parameter θ^{μ}. Algorithms based on deterministic policy gradient, such as DDPG and TD3, produce only a single action $\boldsymbol{a}_t$ given the current state $\boldsymbol{s}_t$

$$\boldsymbol{a}_t = \mu\left(\boldsymbol{s}_t; \theta^{\mu}\right). \tag{13}$$

As shown in Fig. 2, to improve policy exploration, at each state, the APE2 algorithm adopts an enhanced action exploration strategy to efficiently generate an action set $A_t = \{\boldsymbol{a}_t, \boldsymbol{a}_t{}^{(\sigma_m,n)} | 1 \le m \le M; 1 \le n \le N; m, n \in \mathbb{Z}\}$ with diverse and large-scale action candidates

$$\boldsymbol{a}_t{}^{(\sigma_m,n)} = \boldsymbol{a}_t + \mathcal{N}\left(0, \sigma_m^2\right) \tag{14}$$

where σ^2 is the Gaussian variance, M represents the number of different Gaussian noises with diverse scales of variance, and N means the number of times the original action $\boldsymbol{a}_t$ has been explored under each noise.

Remark 3: Deterministic policy gradient-based algorithms can effectively reduce the sample scale compared with stochastic policy-based ones [8], [31]. However, they show limited capacity for exploring the policy. To improve this, algorithms like DDPG add a random noise to the original action $\boldsymbol{a}_t$. However, it does not increase the quantity of available actions, and the randomness may even worsen the performance of the original action. In comparison, by performing enhanced action exploration, the proposed APE2 algorithm can efficiently explore a total number of $M \times N + 1$ action candidates for the agent to select.

B. Hybrid Policy Evaluation

Since the enhanced action exploration provides multiple actions for APE2, it is essential to evaluate and select the optimal one for the agent to execute. The hybrid policy evaluation strategy in the proposed APE2 algorithm consists of both the Q-value, which emphasizes long-term return and the reward, which emphasizes immediate return.

As shown in Fig. 2, the proposed hybrid policy evaluation adopts multiple critics for policy evaluation, and all the estimated Q-values are equally considered. Assuming that K is the number of critics, the integrative Q-value Q_{LTR} is defined as

$$Q_{\mathrm{LTR}} = \frac{1}{\mathrm{K}} \sum_{k=1}^{K} Q_k\left(\boldsymbol{s}_t, \boldsymbol{a}_t; \theta_k^Q\right) \tag{15}$$

where θ_k^Q $(k = 1, 2, \ldots, K)$ is the parameter of each online critic network Q_k. In practice, K can be any positive integer. The impact of different values of K on APE2 will be analyzed in the experiments.

Remark 4: Compared with DDPG, which has only one critic, TD3 and SAC calculate two Q-values by two critics but use only the minimum one for policy evaluation and optimization. Although this approach can prevent the overestimation problem, it assumes that the lower Q-value is always more accurate. Such a policy evaluation considering only a single Q-value could still result in a significant bias. For a DRL agent with multiple critics in a designated task, although the critics have different initial parameters, they share the same optimization target and will converge to near-identical parameters as they approach optimal performance. However, during early training stages, the Q-values they produce can vary significantly, and identifying the most accurate one is challenging without a reference, such as a well-trained critic in the designated task. In this context, compared with favoring and relying solely on a single Q-value, our APE2 algorithm considers the average performance of multiple critics and uses the average Q-value of them to mitigate the bias introduced by any single critic similar to the approach in [24].

Besides Q_{LTR}, APE2 also incorporates the immediate return R_{IR} as part of the policy evaluation

$$R_{\mathrm{IR}} = \sum_{h=1}^{H} r_{t-1+h} \tag{16}$$

where t is the current time step and H is the horizon, indicating the number of time steps over which the reward is considered.

It is worth noting that the proposed parameterized environment allows for the efficient calculation of rewards without

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

real interaction with a simulated or real-world platform. Since the obstacles and the robot are described using mathematical models, including kinematic and spatial geometric equations, both the resulting state $\boldsymbol{s}'$ and the reward $\boldsymbol{r}$ can be computed analytically for any given state–action pair $(\boldsymbol{s}, \boldsymbol{a})$.

Considering both the long-term and immediate returns, the proposed hybrid policy evaluation is defined as

$$V_{\mathrm{HPE}} = \left[\eta \mid 1-\eta\right]\left[Q_{\mathrm{LTR}} \mid R_{\mathrm{IR}}\right]^{\mathrm{T}} \tag{17}$$

where $\eta = \mathrm{clip}(t_c/\mathrm{T}, 0, 1)$. t_c is the cumulative number of policy optimization, and T is a constant. η is clipped to [0,1].

The proposed APE2 algorithm evaluates each action candidate in A_t by the hybrid policy evaluation, and the optimal action is defined as

$$\bar{\boldsymbol{a}}_t = \arg\max_{\boldsymbol{a}\in A_t} V_{\mathrm{HPE}}\big|_{\boldsymbol{s}_t, \boldsymbol{a}_t}. \tag{18}$$

Remark 5: Although Q_{LTR} provides a more unbiased long-term return for policy evaluation, as reported in [23] and [24], the estimation is quite inaccurate in the early training stages due to insufficient training. In contrast, R_{IR} is accurate but shortsighted, focusing merely on the immediate return the agent receives in H time steps from the current time step. According to (17), APE2 leverages the immediate return R_{IR} to provide a more accurate evaluation during the initial training phase. As training proceeds, the weight of R_{IR} decays to zero, transitioning the evaluation to rely solely on Q_{LTR}. Hence, APE2 provides a more accurate and holistic evaluation throughout the training process. While a larger H can make R_{IR} closer to the long-term return, it also increases computational overhead. In practice, setting a small H and let it provide an accurate evaluation in the early training stage is sufficient, since the critics progressively learn to approximate the long-term return defined in (1) as training proceeds. Moreover, APE2 allows efficient generation of diverse action candidates by (14), which enhances the exploration ability compared to the multicritic algorithm in [24].

C. Network Training

Given an interaction data $(\boldsymbol{s}, \boldsymbol{a}, r, \boldsymbol{s}')$, the temporal difference error [7] of each online critic in APE2 is defined as

$$L_k = r + \xi Q'_k\left(\boldsymbol{s}', \mu'\left(\boldsymbol{s}'; \theta^{\mu'}\right)|\theta_k^{Q'}\right) - Q_k\left(\boldsymbol{s}, \boldsymbol{a}|\theta_k^{Q}\right) \tag{19}$$

where $\theta_k^{Q'}$ is the parameter of the target critic network Q'_k and $\theta^{\mu'}$ is the parameter of the target actor network μ'.

Based on (19), the proposed APE2 optimizes each online critic Q_k by the hybrid loss $L_{H,k}$ defined as

$$L_{H,k} = \omega_1 L_k^2 + \left(\frac{1-\omega_1}{K}\sum_{k=1}^{K} L_k^2\right) + \omega_2(\Delta Q)^2 \tag{20}$$

where ω_1 and ω_2 are the weight coefficients. $(\Delta Q)^2 = (Q_k - Q_{\mathrm{LTR}})^2$ is an auxiliary punishment that limits the difference among multiple critics [23].

The proposed APE2 optimizes the online actor μ by

$$\nabla_{\theta^\mu} J \approx \mathbb{E}_\mu\left[\nabla_{\boldsymbol{a}} Q_{\mathrm{LTR}} \nabla_{\theta^\mu}\mu\left(\boldsymbol{s}; \theta^\mu\right)\right]. \tag{21}$$

In addition, the target actor μ' and each target critic Q'_k are optimized by soft update [9].

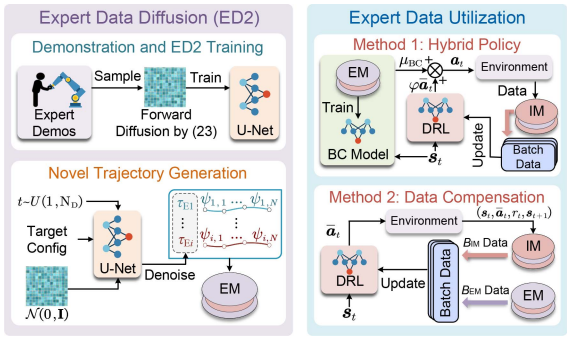


Fig. 3. Frameworks of the ED2 model and two expert data utilization methods. Given a target area, the ED2 model can be efficiently trained using a very limited set of expert demonstrations. The trained model can generate a large number of novel trajectories. The generated expert data can be utilized in two ways: the hybrid policy method creates a synergy between BC and DRL but may not yield optimal trajectories, while the data compensation method enhances DRL training and ensures learning optimal policies.

V. Expert Data Diffusion and Utilization

In this section, we discuss how a diffusion model is leveraged to generate a large-scale trajectory dataset from very limited human demonstrations. Then, two methods for utilizing the diffused dataset are explored to train a robust motion planning policy.

A. ED2 From Limited Demonstrations

As shown in Fig. 3, the diffusion model ϵ_θ of ED2 transforms a trajectory τ from the initial data distribution $\tau_0 \sim \boldsymbol{q}(\tau_0)$ into white Gaussian noise by running a Markovian forward diffusion process

$$\boldsymbol{q}(\tau_t|\tau_{t-1}, t) = \mathcal{N}\left(\tau_t; \sqrt{1-\beta_t}\tau_{t-1}, \beta_t \mathbf{I}\right) \tag{22}$$

where $t \sim U(1, N_{\mathrm{D}})$ is the diffusion time step with uniform distribution U, β_t is the noise scale at time step t [32], and N_{D} is the number of diffusion steps. The value of N_{D} is determined empirically based on the trajectory length (TL) of the expert demonstrations. Assuming that the data τ_0 live in Euclidean space, the distribution of the diffusion process at time step t is Gaussian and can be written in closed form as

$$\boldsymbol{q}(\tau_t|\tau_0, t) = \mathcal{N}\left(\tau_t; \sqrt{\bar{\alpha}_t}\tau_0, (1-\bar{\alpha}_t)\mathbf{I}\right) \tag{23}$$

with $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^{t}\alpha_i$. This allows sampling τ_t without running the forward diffusion process [32].

The inverse (denoising) process of the proposed ED2 transforms Gaussian noise back to the data distribution through a series of denoising steps $p(\tau_{t-1}|\tau_t, t)$. ED2 approximates this posterior distribution with a parameterized Gaussian

$$p_\theta(\tau_{t-1}|\tau_t, t) = \mathcal{N}\left(\tau_{t-1}; \mu_t = \mu_\theta(\tau_t, t), \Sigma_t\right). \tag{24}$$

For simplicity, only the mean of the inverse process is learned, and the covariance is set to $\Sigma_t = \sigma_t^2\mathbf{I} = \tilde{\beta}_t\mathbf{I}$, with $\tilde{\beta}_t = \beta_t(1-\bar{\alpha}_{t-1})/(1-\bar{\alpha}_t)$. Ho et al. [32] suggested that, rather than directly learning the posterior mean, the noise term ϵ could be learned instead via a simplified loss function

$$L(\theta) = \mathbb{E}_{t,\epsilon,\tau_0}[\|\epsilon - \epsilon_\theta(\tau_t, t)\|^2] \tag{25}$$

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

with $t \sim U(1, N_{\mathrm{D}}), \epsilon \sim \mathcal{N}(0, \mathbf{I}), \tau_t = \sqrt{\bar{\alpha}_t}\tau_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$, and $\tau_0 \sim \boldsymbol{q}(\tau_0)$. We encode the diffusion model over trajectories with a temporal U-Net [33], which has proven to be a reasonable architecture for diffusion models over trajectories.

Remark 6: Algorithm 1 presents the pseudocode for training the proposed ED2 model using a very limited set of collision-free trajectories obtained by an expert agent. Due to the small sample size, the training process is highly efficient. The trained model is capable of generating a large number of novel trajectories without the necessity of environmental exploration. Although these trajectories are nonoptimal, they enable the manipulator to move to various poses within the designated workspace. Moreover, by leveraging the nonoptimal dataset, DRL can explore and learn an optimal motion planning policy more efficiently. Compared to existing methods that purely rely on human demonstrations [8] or traditional motion planners [5] to obtain substantial expert data, our method significantly reduces manual effort and offers better stability.

Algorithm 1: Training the ED2 Model.

```
Collision-free trajectories 𝒟, Diffusion model ε_θ,
  noise schedule terms ᾱ_t;
while training is not finished do
    Sample a batch of trajectories:
      τ_0 ~ 𝒟, ε ~ 𝒩(0, I), t ~ 𝒰(1, N_D);
    Compute the diffusion loss function:
    τ_t = √ᾱ_t τ_0 + √(1 − ᾱ_t) ε;
    ℒ(θ) = ‖ε − ε_θ(τ_t, t)‖²;
    Gradient update.
```

With the safety of the provided demonstrations assured, ED2 can be effectively trained to generate collision-free trajectories. The parameterized space verifies the trajectories generated by ED2 according to (7) and (8), ensuring that only collision-free trajectories are utilized. The trajectory τ_{E} generated by the trained ED2 model $\epsilon_{\theta'}$ are converted to $\psi(\boldsymbol{s}, \boldsymbol{a}, r, \boldsymbol{s}')$ format and stored in the expert memory (EM). This process is very efficient using our parameterized task space. We then explore and compare two methods for utilizing the diffused dataset.

B. Synergizing BC and DRL for Motion Planning

In existing works, a popular approach to facilitate DRL training is the formation of a hybrid policy by combining DRL with BC [25], [34]. As shown in Fig. 3, a BC model $\mu_{\mathrm{BC}}(\boldsymbol{s}_t, \theta^{\mu}_{\mathrm{BC}})$ is developed to learn the mapping between state–action pairs at time t (i.e., how s_t maps to a_t) based on the diffused dataset generated by the proposed ED2 model. The loss function is defined as $L_{\mathrm{BC}} = (\boldsymbol{a}_{\mathrm{E}} - \mu_{\mathrm{BC}}(\boldsymbol{s}_{\mathrm{E}}; \theta^{\mu}_{\mathrm{BC}}))^2$, where $(\boldsymbol{s}_{\mathrm{E}}, \boldsymbol{a}_{\mathrm{E}})$ is the labeled action–state pair stored in EM. Since the diffusion step of ED2 is N_{D}, the generated expert trajectory consists of N_{D} joint configuration sequences. Hence, the BC model trained from these trajectory data can only plan motions with fixed N_{D} steps.

For the first N_{D} steps, the action is generated by a hybrid policy wherein the BC model functions as the base policy, and the DRL model serves as the residual policy with a weight φ

$$\boldsymbol{a}_t = \pi(\boldsymbol{s}_t) = \mu_{\mathrm{BC}}(\boldsymbol{s}_t, \theta^{\mu}_{\mathrm{BC}}) + \varphi\bar{\boldsymbol{a}}_t. \tag{26}$$

The weighted sum shown in (26) is a common approach to form a hybrid policy [25], which can create a synergy between BC and DRL. The weight φ should be relatively small. In the hybrid policy, BC provides a base policy to ensure a reasonable performance from the outset, while DRL uses actions with small amplitudes to refine the actions suggested by the base policy, making it more robust to noise disturbances. With the inclusion of the base policy, the tool center point is positioned near the random target after the first N_{D} steps. Thus, we define $\boldsymbol{a}_t = \varphi\bar{\boldsymbol{a}}_t$ for subsequent steps, as using smaller actions allows the agent to effectively explore around the target area and learn to reach the randomly assigned target points. In addition, DRL endows the agent with the ability to keep the tool center point within allowable errors after it reaches the target, even without a termination signal.

C. Diffused Expert Data Compensation Mechanism

Although the hybrid policy proposed in Section V-B can improve both BC and DRL models, its motion planning performance still largely depends on the quality of the demonstrations. Given that neither the human demonstrations nor the diffused dataset are optimal, the hybrid policy often falls short in planning an optimal trajectory.

As shown in Fig. 3, instead of directly cloning the behavior, another popular method is adopting an expert data compensation (DC) mechanism [7], [8], [35] in which the batch data for training DRL agent are sampled from both the interaction memory (IM) pool, which stores the interaction data, and the EM, which stores the diffused expert data. Define B_{IM} as the size of data sampled from IM

$$B_{\mathrm{IM}} = B - B_{\mathrm{EM}} \tag{27}$$

where B is the batch size. When the number of interaction data collected in IM is fewer than B, the value of B is adjusted to match the total number of interaction data available. $B_{\mathrm{EM}} = \mathrm{clip}(\lfloor t/\mathrm{T_B} \rfloor, 0, B^{\max}_{\mathrm{EM}})$ is the size of data sampled from EM, and $\mathrm{T_B}$ is a constant. Unlike the approach in [35], where B_{EM} is set as a constant, we allow B_{EM} to increase progressively from zero to a maximum value as training proceeds. This is to balance B_{EM} with B_{IM}, as there are limited interaction data collected at the beginning of training.

While the agent continuously uses interaction data to update the policy, the proposed DC mechanism gradually supplies additional high-quality data. It is worth noting that although we ensure that the trajectories generated by ED2 are collision-free, the quality of the data in EM does not affect DRL in learning optimal policies. This is because the core capability of DRL models lies in learning optimal planning through experience data in both EM and IM, with the latter constantly providing new interaction data. Previous studies [35] have demonstrated that either small number of high-quality expert demonstrations, or even low-quality but high-coverage demonstrations, can accelerate DRL training. Consequently, compared to the hybrid policy, the DC mechanism not only accelerates the policy searching and learning process of DRL but also ensures that the agent can learn optimal policies, since the action is no longer restricted by BC.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

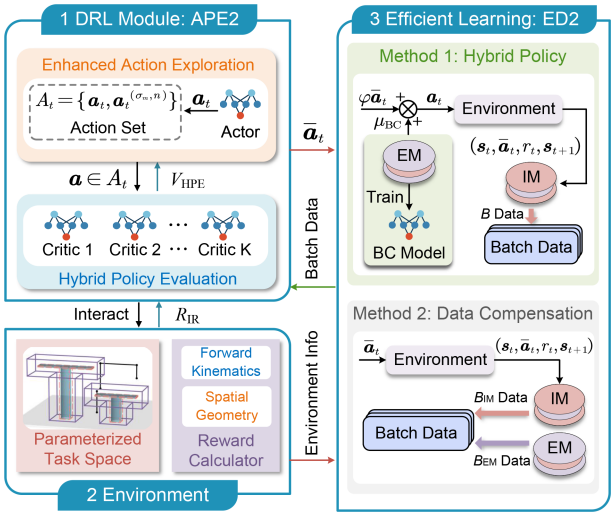


Fig. 4. Overall framework of URPlanner. Our proposed URPlanner consists of three modules: a DRL module where APE2 is applied, an environment module for the parameterized space and reward calculation, and an efficient policy learning module for deploying the ED2 model.

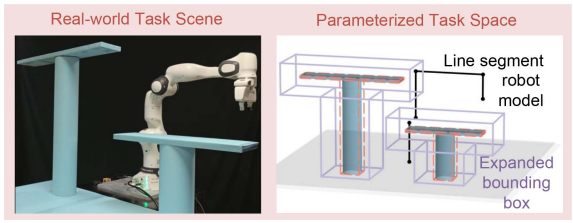


Fig. 5. Real-world scene and the corresponding parameterized space.

TABLE I
PARAMETER SETUPS

Parameter	Value	Parameter	Value	Parameter	Value
learning rate	1e-3	ζ	1	ω_1	0.6
memory capacity	6e4	M	2	ω_2	0.1
soft update rate	0.01	N	3	N_D	80
B	64	T	2e5	φ	0.1
ξ	0.98	H	1	T_B	2e3

VI. Overall Implementation of URPlanner

In this section, the overall framework of our URPlanner is introduced, which integrates the proposed parameterized space, UOAR, APE2 algorithm, and ED2 model.

As shown in Fig. 4, our proposed URPlanner consists of three modules: a DRL module where APE2 is applied, an environment module for the parameterized space and reward calculation, and an efficient policy learning module for deploying the ED2 model. Before training APE2, a certain number of expert trajectories are generated by ED2, converted to $(\boldsymbol{s}, \boldsymbol{a}, r, \boldsymbol{s}')$ format, and stored in the EM. If using the hybrid policy, a BC model is trained based on the expert data. Algorithm 2 provides an overall flow of our URPlanner.

Algorithm 2: Overall Implementation of URPlanner.

Initialize APE2 with an actor and K critics;
Initialize IM and EM, and fill up EM by ED2;
if *use hybrid policy* **then**
  Train a BC model μ_{BC} based on EM;
for *each episode* **do**
  Randomly reset the environment.
  for *each time step* t **do**
    Generate a raw action $\boldsymbol{a}_t$ by Eq. (13);
    Explore an action set A_t by Eq. (14);
    Estimate the value V_{HPE} by Eq. (17);
    Select the optimal action $\bar{\boldsymbol{a}}_t$ defined by Eq. (18);
    if *use hybrid policy* **then**
      Execute action by Eq. (26);
    else: Execute $\bar{\boldsymbol{a}}_t$;
    Calculate the reward $\boldsymbol{r}_t$ by Eq. (12);
    Store $(\boldsymbol{s}_t, \bar{\boldsymbol{a}}_t, \boldsymbol{r}_t, \boldsymbol{s}_{t+1})$ to IM;
    if *use data compensation mechanism* **then**
      Sample B data from IM and EM;
    else: Sample B data from IM only;
    Update online networks by Eqs. (20)-(21);
    Soft update target networks;
    if *the task is implemented* **then**
      Break.

VII. Experimental Results

In this section, the APE2 algorithm, UOAR, and ED2 strategy of our proposed URPlanner are experimentally verified against various state-of-the-art methods. We then compare URPlanner with existing learning-based and traditional planners. It is important to note that URPlanner is trained in the proposed parameterized space. In environments with static obstacle layouts, this parameterized space enables all necessary information for policy training, such as the resulting state and reward after executing an action, to be computed analytically without interacting with a simulated or real-world robot platform.

A. Experimental Setups

The experiments in this section are conducted based on the task scene shown in Fig. 5. The aim is to train a motion planning policy for a seven-DoF Franka Emika Panda robot, enabling it to reach randomly given goal poses above the table without any collision. The algorithms are trained on a laptop equipped with AMD Ryzen 9 5900HX CPU @ 3.30 GHz and 32-GB RAM, with the main parameters shown in Table I. The network architecture for the actor and BC is *state dimension*×256 (ReLu)×256 (ReLu)×*action dimension* (Tanh). The network architecture for the critics is *state+action dimension*×256 (ReLu)×256 (ReLu)×1. The dimension of state and action varies for manipulators with different DoFs. The structure of the U-Net in the ED2 model can be found in [33]. The generated trajectory, represented as joint configuration sequences, is transmitted to the Franka robot via MoveIt, which

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

TABLE II
TRAINING PERFORMANCES OF DDPG ALGORITHM WITH DIFFERENT OBSTACLE AVOIDANCE REWARDS

	Success Rate Per 250 Episodes (%)				Execution Time (s)
	1-250	251-500	501-750	751-1000	per 300 steps
CAR	$0.0\,^{0.0}_{0.0}$	$0.0\,^{0.0}_{0.0}$	$32.3\,^{77.2}_{0.0}$	$95.8\,^{99.6}_{88.8}$	$72.40\,^{73.86}_{71.76}$
COR	$0.0\,^{0.0}_{0.0}$	$6.1\,^{18.0}_{0.0}$	$46.6\,^{66.4}_{0.8}$	$86.4\,^{96.0}_{79.2}$	$72.11\,^{72.64}_{71.60}$
PLR	$0.0\,^{0.0}_{0.0}$	$0.0\,^{0.0}_{0.0}$	$49.1\,^{76.0}_{26.0}$	$99.0\,^{99.6}_{98.4}$	$9.351\,^{9.526}_{9.119}$
UOAR	$0.0\,^{0.0}_{0.0}$	$0.0\,^{0.0}_{0.0}$	$56.3\,^{78.0}_{33.2}$	$96.4\,^{98.8}_{94.4}$	$\mathbf{0.776}\,^{0.785}_{0.768}$

Annotation: The data are in the format of $\text{mean}^{\max}_{\min}$. The success rate over a given episode interval $[a, b]$ is defined as $c/(b-a+1) \times 100\%$, where c represents the number of episodes in which the agent successfully completes the task. This metric reflects how fast the agent finds and learns a stable policy for completing the given task. The execution time counts in all the necessary procedures for a complete interaction between the agent and the environment, including the time for state observation, action calculation and execution, reward calculation, as well as communication.

TABLE III
QUANTITATIVE AND QUALITATIVE ANALYSES OF DIFFERENT OBSTACLE AVOIDANCE REWARDS

	CAR	COR	PLR	UOAR
Testing success rate	100.0%	100.0%	100.0%	100.0%
Trajectory length	$1.174^{1.203}_{1.136}$	$1.103^{1.117}_{1.086}$	$1.186^{1.214}_{1.169}$	$1.053^{1.072}_{1.036}$
Minimum distance independent	×	×	×	✓
Platform-agnostic	×	×	✓	✓
Efficiency	very low	very low	high	very high
Practical value	normal	normal	high	very high

Annotation: The testing success rate is calculated based on 50 trials. The trajectory length is in the format of $\text{mean}^{\max}_{\min}$. The practical value evaluates the time cost and effectiveness in real-world applications.

subsequently sends actuation commands to the robot's joints. To control the robot's speed, the maximum velocity and acceleration scaling factors are set to 0.25.

B. Training Performance Evaluation

1) Evaluation of UOAR: First, we compare the training performance of the proposed UOAR with other obstacle avoidance rewards reported in recent research. For each method, we substitute only (10) of UOAR by other obstacle avoidance rewards. Referring to [7, eqs. (16) and (18)], COR is a quadratic function based on the precise MD between the manipulator and obstacles, consisting of an attraction component and a repulsive component. CAR is a linear function proportional to the MD, which was proposed in [22] and modified in [7]. Its setup can be found in [7, eq. (21)]. Both methods have been proven effective in collision-free robotic motion planning. In [7], COR and CAR are non-platform-agnostic methods, and the DRL agent is trained on CoppeliaSim.

It is worth noting that the MD can also be approximately obtained in the proposed parameterized space through proper abstraction. This allows MD-based approaches, such as CAR and COR, to be designed in a platform-agnostic manner. Prior studies like [26] and [30] simplified the acquisition of the MD by calculating the distance between an obstacle's center point and the manipulator's link. Based on this, they introduced a center-point-to-line reward to learn obstacle avoidance. However, this method oversimplifies obstacles by modeling them as mass points, leading to a significant loss of spatial information. As a result, it struggles to handle the complex task scenarios addressed in our work. Thus, we make some improvements to it and design a points-to-line reward (PLR). PLR refines the bounding box representation by subdividing its surface into multiple points at equal intervals. The MD is then calculated by identifying the shortest distance between all points on the bounding box and the manipulator's line segments. This MD is used by PLR with the same reward function as CAR. Both PLR and the proposed UOAR are platform agnostic and trained in the parameterized space.

In this experiment, DDPG is trained using different obstacle avoidance rewards. As shown in Table II, we use the success rate (SR) in training to reflect how fast the DRL agent with different obstacle avoidance rewards finds and learns a stable policy for completing the given task. Between the 1st to 750th training episodes, the SRs (averaged over four random seeds) of the four methods are relatively low, suggesting that the agent has not yet learned a stable policy for completing the tasks and requires further training. In contrast, between the 751st and 1000th episodes, the training SRs of CAR, PLR, and UOAR reach nearly 100%, indicating that the policies have converged to a desired state, and the training process can be terminated.

Although COR achieves an SR of 6.1% between 251st and 500th episodes, compared to zero for the other three reward functions, it performs slightly worse in the subsequent training period. According to the SRs in Table II and Fig. 6(c), PLR and the proposed UOAR exhibit similar performance in learning a stable policy for completing the tasks, and both of them slightly outperform CAR and COR.

Besides the efficiency in converging to a stable policy, we also compare the execution time required for training across different methods to evaluate their time cost. In this work, we propose a DRL-based motion planning paradigm that is highly cost-effective in both training and deployment. This efficiency arises from the proposed parameterized space and the MD-independent reward function, UOAR.

First, our platform-agnostic method uses parameterized space to represent the environment, enabling all necessary information for training the DRL agent to be computed analytically. In contrast, existing DRL-based motion planners, such as those in [5], [7], [8], [9], [11], [25], and [26], rely on interactions with specific simulated or real-world robotic platforms to obtain training data. These methods incur significant time costs due to communication between the agent and the platform to send states and actions, and the waiting time for action execution to retrieve the resulting state and reward. As shown in Table III, CAR and COR are non-platform-agnostic methods, while PLR and UOAR are platform-agnostic methods based on the proposed parameterized space. Table II indicates that CAR and COR take over 70 s to complete 300 interactions, whereas PLR and UOAR require only 9.351 and 0.776 s, respectively. This substantial reduction in execution time demonstrates the significant advantage of our platform-agnostic approach in minimizing training time.

Second, the proposed UOAR strikes a balance between simplifying reward calculations and preserving sufficient spatial information about the manipulator and obstacles, while also maintaining satisfactory training SRs. Existing DRL-based planners rely heavily on computing the MD between the robot and

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

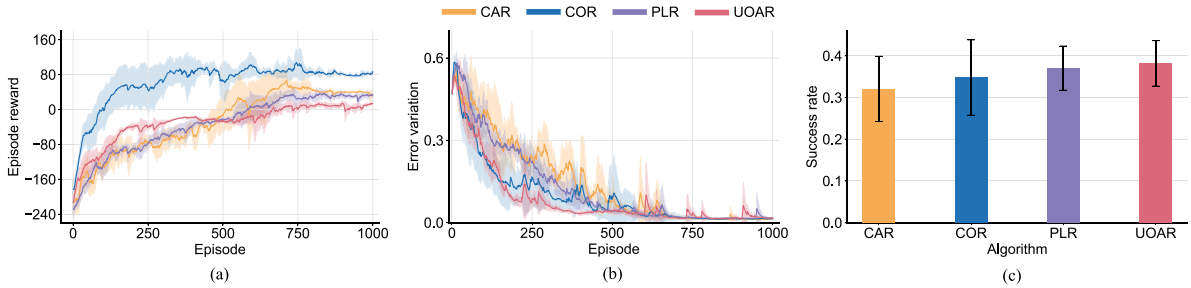


Fig. 6. Training performances of DDPG algorithms based on different obstacle avoidance rewards. The shaded region and the error bar represent the standard deviation of the average evaluation over four random seeds. (a) Episode reward. (b) Error variation. (c) Total training success rate.

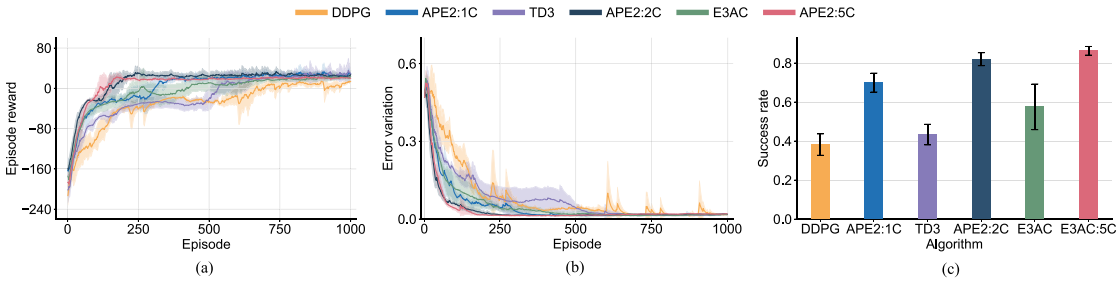


Fig. 7. Training performances of different DRL algorithms with UOAR. The shaded region and the error bar represent the standard deviation of the average evaluation over four random seeds. (a) Episode reward. (b) Error variation. (c) Total training SR.

obstacles to calculate rewards, which is more time intensive. The PLR and UOAR in Table II provide a fair comparison on the time cost for calculating rewards, as both methods are trained in the parameterized space using the same DRL algorithm, differing only in their reward calculation methods. In the scenarios with four obstacles, PLR requires 9.351 s for 300 interactions, while UOAR requires only 0.776 s. This difference, stemming solely from the reward computation process, underscores the advantage of UOAR in significantly reducing the time cost of training. Moreover, our method shows satisfactory scalability in terms of computational efficiency. In the scenario with eight obstacles, the execution time (for 300 interactions) increases to $0.799^{0.824}_{0.786}$ s for UOAR and $16.87^{17.04}_{16.54}$ s for PLR. In an environment with 16 obstacles, UOAR completes the same number of interactions in $0.861^{0.886}_{0.840}$ s, while PLR requires $32.77^{33.36}_{32.45}$ s. These results clearly show that as the number of obstacles increases, the computational advantage of UOAR becomes even more pronounced.

In addition, more analysis of the four obstacle avoidance rewards is provided in Table III. In conclusion, the proposed UOAR demonstrates superior practical value than other counterparts, as it is MD independent, platform agnostic, and highly efficient in both training and deployment. Although UOAR relies on certain abstractions of real-world environment, such as padding obstacle's bounding box, it can be seen that a proper expansion does not result in an increase in TL. In Section VII-C, more comparisons regarding the quality of trajectories planned by these rewards will be presented in real-world applications. In the subsequent experiments, all the algorithms will be trained using UOAR unless otherwise specified.

2) Evaluation of APE2: Second, the proposed APE2 algorithm is applied to various DRL benchmarks, including DDPG, TD3, and E3AC, to evaluate its superiority and universality.

TABLE IV
TRAINING SUCCESS RATE AND STABILITY OF DIFFERENT DRL ALGORITHMS

	Success Rate Per 250 Episodes (%)				Reward Fluctuation (RF)
	1-250	251-500	501-750	751-1000	
DDPG	$0.0^{0.0}_{0.0}$	$0.0^{0.0}_{0.0}$	$56.3^{78.0}_{33.2}$	$96.4^{98.8}_{94.4}$	$45.2^{52.4}_{34.3}$
APE2:1C	$2.5^{10.0}_{0.0}$	$79.0^{99.2}_{56.0}$	$99.4^{100.0}_{98.8}$	$99.0^{100.0}_{98.0}$	$39.4^{44.8}_{36.3}$
TD3	$0.0^{0.0}_{0.0}$	$0.2^{0.4}_{0.0}$	$74.2^{92.4}_{37.6}$	$98.8^{100.0}_{96.8}$	$44.4^{49.7}_{39.7}$
APE2:2C	$28.4^{44.0}_{7.2}$	$100.0^{100.0}_{100.0}$	$99.6^{100.0}_{99.2}$	$99.7^{100.0}_{99.2}$	$31.9^{34.7}_{27.7}$
E3AC	$0.0^{0.0}_{0.0}$	$40.4^{94.0}_{0.0}$	$90.5^{100.0}_{62.0}$	$99.2^{99.6}_{98.8}$	$34.1^{40.1}_{27.5}$
APE2:5C	**45.7** $^{60.4}_{36.8}$	$100.0^{100.0}_{100.0}$	$99.9^{100.0}_{99.6}$	$99.5^{100.0}_{99.2}$	**30.4** $^{34.8}_{26.7}$

Annotation: The data are in the format of $\text{mean}^{\text{max}}_{\text{min}}$. APE:1C, APE2:2C, and APE2:5C differ only in the number of critics (1, 2, and 5, respectively). For a DRL algorithm, the RF represents the standard deviation of episode rewards during training, capturing the extent of their fluctuations. A smaller RF value indicates that the DRL algorithm is more stable in training.

DDPG is a typical DRL algorithm based on deterministic policy gradient, which performs well in handling high-dimensional and continuous spaces. However, it often exhibits unsatisfactory learning efficiency and stability, especially in complex tasks such as collision-free motion planning for redundant manipulators. As shown in Table IV, the training SR is zero in the first 500 episodes. Although DDPG successfully learns a policy to complete the tasks during the 501st to 1000th episodes, Fig. 7(a)–(b) reveals that the policy does not converge until approximately the 650th episode. Specifically, episode rewards are a key metric for evaluating the training performance of DRL, reflecting how effectively the agent maximizes the reward signal over time. Notably, the episode rewards for DDPG remain relatively lower

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

than those of its counterparts after convergence. Furthermore, significant reward fluctuations (RFs) are observed around the 750th and 900th episodes. While minor fluctuations are normal postconvergence, substantial fluctuations often indicate instability in training. To quantify this, we calculate the standard deviation of episode rewards during training as a measure of RF. The RF for DDPG (averaged over four seeds) is 45.2 and is the largest among all compared methods.

By applying APE2 to DDPG, we create its APE2 variant, which has only one critic (denoted as APE2:1C). Hence, APE2:1C relies on a single Q-value to estimate the long-term return. APE2:1C explores diverse action candidates by (14), evaluates their values by the proposed hybrid policy evaluation, and executes the optimal one according to (18). Compared to DDPG, APE2:1C achieves an SR (averaged over four random seeds) of 79.0% during the 251st to 500th training episodes and converges around the 350th episode. Moreover, the average RF during training is reduced to 39.4, indicating enhanced stability. As shown in Fig. 7(a), the episode rewards are relatively stable after the policy converges, further demonstrating the improved training performance of APE2:1C compared to DDPG.

TD3 has two critics but uses only the minimum Q-value for policy optimization, which can mitigate the overestimation problem of DDPG. However, as shown in Table IV, although the SR (averaged over four seeds) during 501st to 700th training episodes improves by 17.9% compared to DDPG, the total SR throughout training remains significantly lower than that of the three APE2 variants (APE2:1C, APE2:2C, and APE2:5C). Furthermore, TD3 assumes that the lower Q-value is always more accurate. Such a policy evaluation considering only a single Q-value could still result in a significant bias. Consequently, the RF of TD3 is very close to that of DDPG. In contrast, the APE2 variant of TD3, which employs two critics (denoted as APE2:2C), uses both Q-values during policy evaluation, resulting in notable training performance. As shown in Fig. 7(a) and Table IV, the policy converges in approximately 250 episodes and maintains an SR of nearly 100% thereafter. In addition, the RF decreases to only 31.9, further demonstrating the stability and effectiveness of the proposed policy exploration and evaluation approach.

E3AC is one of the latest deterministic policy gradient-based algorithms, utilizing five Q-values for more unbiased policy evaluation. As shown in Table IV and Fig. 7(a), E3AC outperforms DDPG and TD3 in terms of SR, convergence speed, and stability, both of which rely on a single Q-value. However, the total SR throughout training remains significantly lower than that of the three APE2 variants. This limitation arises because, in the early stages of training, the Q-value estimation by the critics is still inaccurate due to insufficient training data and iterations. In contrast, the APE2 variant of E3AC, which incorporates five critics (denoted as APE2:5C), achieves a 45.7% SR during the 1st to 250th episodes. Moreover, it exhibits the lowest RF at only 30.4. This outstanding performance can be attributed to the inclusion of the immediate return R_{IR} in policy evaluation, which enhances accuracy during the early training stages.

In addition, we evaluate the policies learned by the aforementioned DRL algorithms by testing them with seen and unseen goal poses after training is completed. As shown in Table V, all algorithms achieve a 100% SR for reaching 50 random goal positions within the trained target area without collision. To assess generalization capabilities, two regions along the y-axis are tested for unseen goal positions: the region within 5 cm of the trained area, and the region 5–10 cm away from the trained area.

TABLE V
TESTING SUCCESS RATE OF DIFFERENT DRL ALGORITHMS

	DDPG	APE2:1C	TD3	APE2:2C	E3AC	APE2:5C
seen	100%	100%	100%	100%	100%	100%
unseen ($\leq$5 cm)	100%	100%	100%	100%	100%	100%
unseen (5 to 10 cm)	60%	100%	58%	100%	96%	100%

Annotation: The testing success rate is calculated based on 50 trials. Two regions along the y-axis are tested for unseen goal positions: the region within 5 cm of the trained area, and the region 5 to 10 cm away from the trained area. In each region, 50 new target poses are uniformly distributed.

TABLE VI
TRAINING PERFORMANCES OF DIFFERENT DATA ACQUISITION AND UTILIZATION METHODS

	Success Rate Per 250 Episodes (%)			
	1-250	251-500	501-750	751-1000
E3AC	$0.0\,^{0.0}_{0.0}$	$40.4\,^{94.0}_{0.0}$	$90.5\,^{100.0}_{62.0}$	$99.2\,^{99.6}_{98.8}$
E3AC:BC	$16.8\,^{35.6}_{5.6}$	$97.6\,^{100.0}_{96.0}$	$98.3\,^{99.2}_{97.2}$	$97.0\,^{97.6}_{96.0}$
E3AC:DC-RL	$48.6\,^{50.8}_{45.2}$	$99.9\,^{100.0}_{99.6}$	$99.5\,^{99.6}_{99.2}$	$99.6\,^{100.0}_{98.8}$
E3AC:DC-ED2	$40.2\,^{42.8}_{36.8}$	$99.9\,^{100.0}_{99.6}$	$99.3\,^{100.0}_{98.4}$	$99.8\,^{100.0}_{99.2}$
APE2	$45.7\,^{60.4}_{36.8}$	$100.0\,^{100.0}_{100.0}$	$99.9\,^{100.0}_{99.6}$	$99.5\,^{100.0}_{99.2}$
APE2:BC	$53.0\,^{66.8}_{40.0}$	$99.9\,^{100.0}_{99.6}$	$98.0\,^{98.4}_{97.6}$	$99.1\,^{100.0}_{98.0}$
APE2:DC-RL	$67.8\,^{71.2}_{65.2}$	$99.9\,^{100.0}_{99.6}$	$99.8\,^{100.0}_{99.6}$	$99.3\,^{100.0}_{98.0}$
APE2:DC-ED2	$63.7\,^{66.8}_{61.2}$	$99.9\,^{100.0}_{99.6}$	$99.8\,^{100.0}_{99.6}$	$99.4\,^{99.6}_{99.2}$

Annotation: The data are in the format of $\mathrm{mean}^{\max}_{\min}$. All the algorithms achieve a 100% testing success rate based on 50 trials.

In each region, 50 unseen target poses are uniformly distributed for evaluation. Results show that for unseen goals within 5 cm of the trained area, all algorithms maintain a 100% SR. However, for unseen goals 5–10 cm away, benchmark algorithms such as DDPG and TD3 exhibit a significant drop in SRs. In contrast, the proposed APE2 algorithm, regardless of the number of critics, consistently achieves a 100% SR, demonstrating its superior generalization ability.

Based on the aforementioned discussion, it can be concluded that the proposed APE2 algorithm can effectively improve the performance of various DRL benchmarks by enhancing policy exploration and evaluation. According to Fig. 7, APE2:1C, APE2:2C, and APE2:5C are the top three algorithms in terms of training SR. Their performances are relatively more stable across different seeds as indicated by the narrower shaded region representing the standard deviation. By further comparing them, we observe that the training performance improves with an increased number of critics. Therefore, we use APE2:5C for subsequent experiments and simply denote it as APE2.

3) Evaluation of ED2 and Its Utilization: Finally, we conduct a series of experiments to evaluate the impact of the diffused dataset (containing 800 expert trajectories) generated by the ED2 model on the training efficiency of DRL benchmarks. As shown in Table VI, we perform a set of four experiments based on both E3AC and APE2. In each set, E3AC:BC and APE2:BC represent the hybrid policy discussed in Section V-B, which creates a synergy between BC and DRL. DC-ED2 and DC-RL represent the expert DC mechanism discussed in Section V-C, where the expert trajectory data in the EM are generated by two sources: the proposed ED2 model (DC-ED2) and a well-trained

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

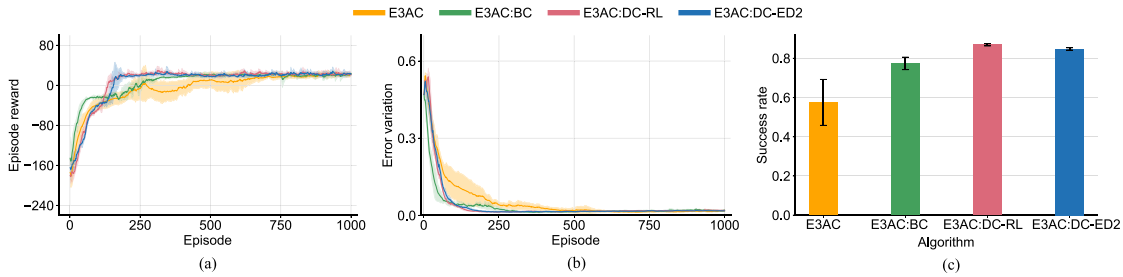


Fig. 8. Comparison of the hybrid policy and DC mechanism in facilitating the training of the E3AC algorithm. The shaded region and the error bar represent the standard deviation of the average evaluation over four random seeds. (a) Episode reward. (b) Error variation. (c) Total training SR.

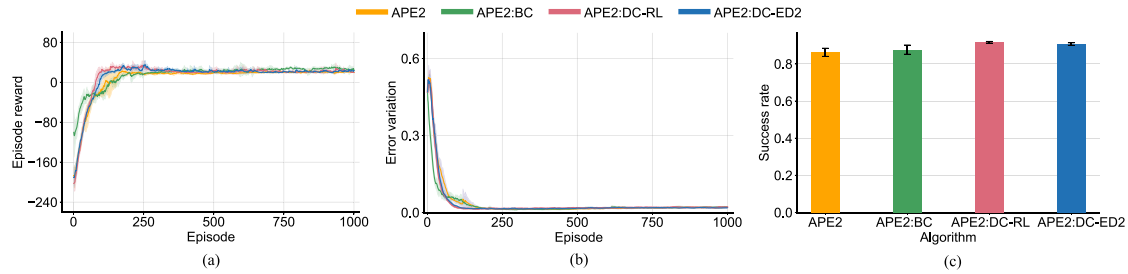


Fig. 9. Comparison of the hybrid policy and DC mechanism in facilitating the training of APE2 algorithm. The shaded region and the error bar represent the standard deviation of the average evaluation over four random seeds. (a) Episode reward. (b) Error variation. (c) Total training SR.

RL model (DC-RL). Thus, DC-RL contains optimal trajectory data, whereas DC-ED2 includes nonoptimal data.

From the perspective of efficiency to train a stable policy, as shown in Table VI, both E3AC and APE2 have already achieved satisfactory training SRs. By further combining a BC model trained using the diffused dataset generated by ED2 to form a hybrid policy, E3AC:BC and APE2:BC have promoted the SR to 16.8% and 53.0% in the first 250 training episodes, respectively. Compared to the hybrid policy, the DC mechanism shows better improvements in training performance, achieving an SR of 40.2% for E3AC:DC-ED2 and 63.7% for APE2:DC-ED2 during this training period. Specifically, it takes only about 100 training episodes for APE2:DC-ED2 to converge to a stable motion planning policy according to Fig. 9(a).

To further evaluate the effectiveness of the proposed ED2 in accelerating training with high-quality but nonoptimal expert data, we use a well-trained RL model to provide optimal expert data for DC. As shown in Table VI, both E3AC:DC-RL and APE2:DC-RL achieve the highest success rates in the first 250 training episodes, compared with the corresponding benchmarks. However, using DC-RL creates an idealized condition where the demonstrations are optimal, which is unrealistic given that the trajectories demonstrated by an expert are typically nonoptimal. In contrast, ED2 is more realistic and practical, allowing DRL to search for and learn an optimal policy more efficiently with the aid of some high-quality but nonoptimal expert data. As shown in Figs. 8(a), and 9(a), the episode rewards of E3AC:DC-ED2 and APE2:DC-ED2 converge to values similar to E3AC:DC-RL and APE2:DC-RL, respectively. This indicates that DRL algorithms trained with the help of either optimal or nonoptimal expert data can learn comparable policies. Furthermore, as depicted in Figs. 8(c) and 9(c), the total SRs of E3AC:DC-ED2 and APE2:DC-ED2 throughout training

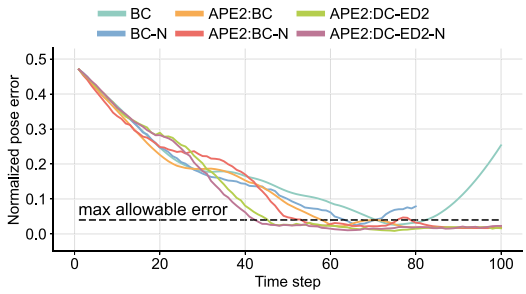


Fig. 10. Robustness analyses among different uses of the proposed ED2 strategy. The robustness is evaluated from two aspects: the ability to keep the tool center point within allowable errors after reaching the goal pose, and the ability to withstand noise disturbances.

are only slightly lower than those achieved using DC-RL (i.e., E3AC:DC-RL and APE2:DC-RL). According to Table VI, this very slight reduction in SRs is mainly reflected in the early training stages.

Moreover, the robustness of well-trained BC, APE2:BC, and APE2:DC-ED2 models based on ED2 has been evaluated from two aspects: the ability to keep the tool center point within allowable errors after reaching the goal pose and the ability to withstand noise disturbances. Fig. 10 shows the variation of normalized pose error for each model in motion planning, given a fixed goal pose. Since the diffusion step of ED2 is 80, BC can only plan the motion with fixed 80 steps. Consequently, the pose error of BC increases continuously from the 80th time step onward, as it cannot maintain the tool center point within allowable errors. In addition, when Gaussian noise with a variance of 0.4 is added to the action space (see BC-N), a distribution shift phenomenon is observed, with the error of

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

BC-N exceeding the maximum allowable range by the end of the planning task. Compared with BC, the proposed hybrid policy (see APE2:BC) uses only about 60 steps to reduce the pose error to allowable ranges and maintains it within this range for subsequent steps. Furthermore, the hybrid policy shows good robustness against noise disturbances (see APE2:BC-N).

Comparing APE2:BC and APE2:DC-ED2, it can be inferred that the proposed DC mechanism helps DRL learn better motion planning policies. Specifically, the pose error of APE2:DC-ED2 is reduced to within the allowable range in about 50 steps. The subsequent error fluctuation remains minimal and stable, even under noise disturbances (see APE2:DC-ED2-N). In general, pure DRL-based models exhibit greater robustness than BC-based models. Since APE2:DC-ED2 is entirely independent of BC, its motion planning performance is not constrained by the quality of the provided expert demonstrations.

Based on the experimental results presented in this section, the proposed ED2 model can effectively generate substantiate expert trajectory data, which can significantly enhance DRL training when utilized with appropriate data integration methods. Moreover, the DC mechanism outperforms the hybrid policy learning approach in terms of both training efficiency and trajectory generation quality. Consequently, we adopt the DC mechanism as the primary technique for efficient policy learning in our proposed URPlanner.

The results in Fig. 10 also demonstrate the capacity of our proposed URPlanner to handle tracking errors effectively. In this work, the trained policy is rolled out in the parameterized environment in a closed-loop feedback manner to generate trajectories (represented as joint configuration sequences). As shown in Fig. 10, even if the actual robot configurations deviate from the originally commanded configurations because of the noise disturbances introduced to the action space at each time step, URPlanner can continue to plan and accomplish the assigned tasks with satisfactory performance. After the trajectory is planned in a closed-loop manner and its safety is verified, the joint configuration sequence is transmitted to the real-world robot for execution in an open-loop manner. In such cases, if the robot fails to reach the commanded configuration at any step due to disturbances, the error is automatically corrected as it moves toward the next joint configuration in the sequence. Moreover, a feedback loop can be implemented in URPlanner to replan subsequent trajectories at a specific frequency. For instance, URPlanner can periodically read the actual joint angles of the robot every few steps to calculate the current state and update all the configurations yet to be tracked. The replanning can be accomplished within a few milliseconds by URPlanner, as demonstrated in the next section.

C. Application Instances

In this section, our proposed URPlanner is compared with other existing traditional and learning-based motion planners to verify its superiority in trajectory generation.

1) Comparisons Among Existing Planners: As shown in Fig. 11, we evaluate the quality of trajectories planned by existing planners for a given goal pose. RRT and RRT* are selected as representatives of traditional methods, with a maximum planning time of 4 s. DDPG:CAR and DDPG:COR, as proposed in [7], are powerful learning-based planners. The trajectories for these four methods are generated using a virtual twin system [9] established in CoppeliaSim and then applied to real robots, as

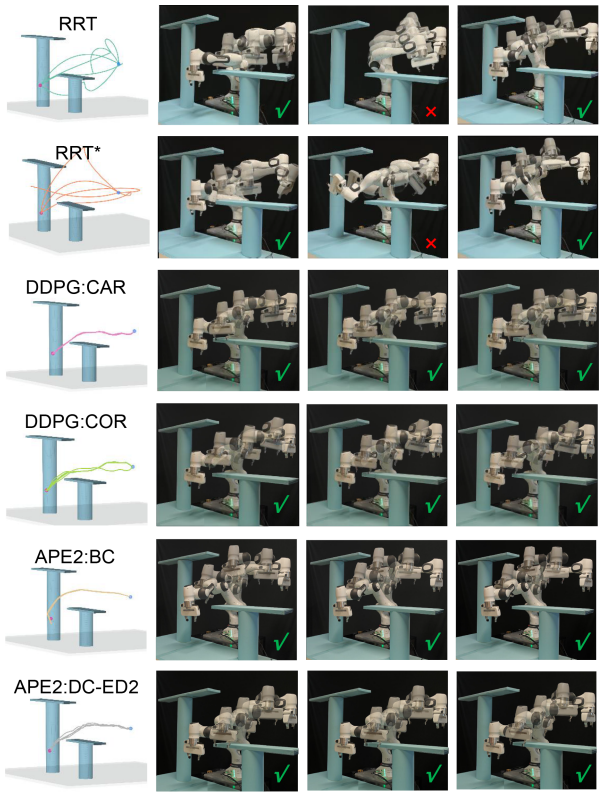


Fig. 11. Trajectory generation performance of each planner given a fixed goal pose. Column 1 contains all the five trajectories. Columns 2–4 are three selected real-world examples. The green tick and red cross indicate success and failure, respectively.

they are not platform agnostic. Specifically, for DDPG:CAR and DDPG:COR, the policy is rolled out in the virtual twin system in a closed-loop feedback fashion. The agent observes the current state, computes an action using the trained policy, and transitions to the next state. This process continues iteratively until the goal is achieved. Hence, the generation cost (GC) depends on both the simulation runtime and the inference time of the policy.

In our evaluation of URPlanner, we assess the performance of both APE2:BC and APE2:DC-ED2 to determine which approach offers better results. As platform-agnostic methods, they use the proposed parameterized space to represent the environment with static obstacle layouts, enabling all necessary information for trajectory generation to be computed analytically. Thus, trajectory generation is reduced to a direct computation of states and actions, without requiring a simulated robot platform. Here, the GC depends solely on the policy's inference time, significantly lowering the time cost for generating feasible trajectories.

The experiment is repeated five times for each planner, and only successful trials are counted when calculating the TL and the MD between the robot and obstacles during motion. Note that the MD data in Table VII are obtained using the MD module in CoppeliaSim by verifying the generated trajectories in the virtual twin system.

As shown in Table VII, the training cost (TC) of RRT and RRT* is zero, as they do not need training. However, the performance is highly unstable. For instance, the GC, TL, and MD for RRT vary significantly, ranging from 0.281 to 4.0 s, 1.071 to 1.589 m, and 0.09 to 5.34 cm, respectively. This instability is

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PERFORMANCE COMPARISONS OF DIFFERENT MOTION PLANNERS

Category	Planner	Training Cost	Generation Cost (s)	Success Rate	Trajectory Length (m)	Minimum Distance (cm)	Score	IK-free	Platform Agnostic	Stability
Traditional Planner	RRT	0	$1.937\,^{4.0}_{0.281}$	80.0%	$1.340\,^{1.589}_{1.071}$	$2.62\,^{5.34}_{0.09}$	3.081	×	×	normal
	RRT*	0	$4.0\,^{4.0}_{4.0}$	60.0%	$1.457\,^{1.676}_{1.102}$	$1.68\,^{2.74}_{1.15}$	1.986	×	×	normal
Learning Based	DDPG:CAR	11h44min	$6.34\,^{6.44}_{6.24}$	100.0%	$1.153\,^{1.164}_{1.140}$	$5.22\,^{5.41}_{5.06}$	2.802	✓	×	good
	DDPG:COR	9h48min	$6.47\,^{6.74}_{6.26}$	100.0%	$1.098\,^{1.140}_{1.080}$	$5.36\,^{6.03}_{5.07}$	3.136	✓	×	good
	APE2:BC	13min4s	$0.212\,^{0.218}_{0.207}$	100.0%	$1.285\,^{1.296}_{1.277}$	$4.92\,^{5.13}_{4.68}$	4.293	✓	✓	good
	APE2:DC-ED2	**8min49s**	$\mathbf{0.060}\,^{0.061}_{0.060}$	**100.0%**	$\mathbf{1.098}\,^{1.108}_{1.089}$	$\mathbf{5.47}\,^{5.92}_{5.11}$	**4.987**	✓	✓	good

Annotation: The data of planning time, trajectory length, and minimum distance between the manipulator and obstacles are in the format of $\text{mean}^{\text{max}}_{\text{min}}$. The trajectories are generated in a simulated or parameterized environment to ensure safety, and subsequently sent to the real-world robot in an open-loop fashion. The generation cost refers to the time required by various methods to produce a feasible trajectory for a given task. For non-platform-agnostic methods, this cost includes both the simulation runtime and the policy inference time. In contrast, for our platform-agnostic methods, it depends solely on the policy inference time.

attributed to the infinite IK solutions for redundant manipulators and the inherent randomness of the tree-based search process, leading to occasional failures during the five trials. Notably, RRT* stabilizes the GC at the maximum value of 4 s, attempting to asymptotically find the optimal solution. However, the random process often renders the 4-s planning time insufficient for identifying a solution, resulting in the lowest SR of only 60% . Similar to RRT, the TL and MD of RRT* also vary significantly. Moreover, even for the successful trajectories generated by RRT and RRT*, the MD is often too small to guarantee safety during motion.

Compared to RRT and RRT*, a major drawback of learning-based methods is their high TC. We train the four planners until their policies converge to a stable state. As shown in Table VII, training DDPG:COR and DDPG:CAR takes about 10 and 12 h, respectively. This lengthy training period is due to the need for interactions with simulated or real-world platforms to acquire necessary environmental information and their reliance on precise MD calculations, as discussed in Section VII-B1. For the same reason, both planners require over 6 s to generate a trajectory. In contrast, the APE2:BC and APE2:DC-ED2 models of our URPlanner, as platform-agnostic methods, require only a few minutes for training and a few milliseconds for trajectory generation. This efficiency is achieved using the proposed parameterized space and MD-independent reward. Particularly, the APE2:DC-ED2 model learns a stable policy in less than 9 min and requires only 0.06 s to generate an entire trajectory.

Although the four learning-based motion planners incur certain TCs, they demonstrate good stability and satisfactory trajectory quality. This is because DRL enables the agent to learn an optimal policy to maximize the return (episode reward). As shown in Table VII, all of them achieved a 100% SR in all the tests. It is also noteworthy that, compared to traditional planners, the average length of the trajectories planned by these models is shorter, while the corresponding MD (between the manipulator and the obstacles throughout the motion process) is larger, indicating superior trajectory quality and safety. Moreover, the TL and MD of the five trajectories planned by each learning-based method exhibit significantly lower variance compared to traditional planners, indicating better stability in motion planning. In addition, the four planners are designed to be IK-free, allowing them to be applied to arbitrary manipulators without requiring IK planning.

By further comparing among the four DRL-based models, it is worth noting that the trajectories planned by APE2:BC show the largest TL of 1.285 m. In addition, although the MD is satisfactory, it is still slightly smaller than the other models. This is because directly incorporating a nonoptimal BC policy in the action $\boldsymbol{a}_t$ can also restrict the overall behavior of the hybrid policy, even though it helps accelerate the training of DRL algorithms. In comparison, using the DC mechanism not only brings greater promotion to the training performance (as discussed in Section VII-B3), but also ensures that the DRL agent can effectively search for and learn an optimal policy. As shown in Table VII, the trajectories planned by APE2:DC-ED2 model exhibit the shortest average TL while maintaining the largest average MD.

As described in Section III-A, a safety offset a_{o} is introduced to maintain a safe distance from obstacles throughout the motion. As shown in Table VII, with $a_{\text{o}} = 5$ cm, the trajectories planned by the APE2:DC-ED2 have a TL of $1.098^{1.108}_{1.089}$ m and an MD of $5.47^{5.92}_{5.11}$ cm. However, excessively large safety offsets can overconstrain the motion planner. To investigate this tradeoff, we analyzed the effect of different a_{o} on both TL and MD. When $a_{\text{o}} = 0$ cm, the trajectories planned by the APE2:DC-ED2 have a TL of $1.053^{1.058}_{1.042}$ m and an MD of $1.58^{1.91}_{1.00}$ cm. When $a_{\text{o}} = 2.5$ cm, the TL and MD are $1.069^{1.077}_{1.060}$ m and $2.75^{3.48}_{1.68}$ cm, respectively. When $a_{\text{o}} = 8.0$ cm, the planner fails to identify a solution.

These results indicate that setting a_{o} too small (e.g., 0 cm) can yield slightly shorter trajectories but compromises safety, as even minor disturbances may lead to collisions during trajectory tracking. Conversely, an overly large a_{o} (e.g., 8.0 cm) may render the task infeasible, as no robot configuration would exist that satisfies the constraints. Our empirical analysis suggests that a safety offset in the range of 2.5–5 cm provides a practical balance between safety and task feasibility. Based on these findings, and consistent with the setup in prior works [9], we adopt a 5-cm safety offset in our framework.

In Fig. 12, we score the overall performance of each motion planner by synthesizing the aforementioned criteria: TC, GC, SR, TL, and MD. The values in Table VII are scaled to the range [0,1] using min–max normalization, except for the SR. For all criteria except SR and MD, the score is calculated as (1−normalized value). For MD, the normalized value is used directly as the score. Consequently, the APE2:DC-ED2 model of URPlanner achieves the highest score of 4.987,

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
PERFORMANCE OF DIFFERENT MOTION PLANNERS WITH VARIED GOAL POSES

	point 1				point 2				point 3			
Planner	**SR**	**TL (m)**	**MD (cm)**	**Score**	**SR**	**TL (m)**	**MD (cm)**	**Score**	**SR**	**TL (m)**	**MD (cm)**	**Score**
RRT	100.0%	$1.147\,^{1.391}_{0.956}$	$2.76\,^{4.33}_{0.55}$	1.625	60.0%	$0.982\,^{0.999}_{0.971}$	$4.35\,^{7.74}_{0.41}$	1.582	60.0%	$0.948\,^{1.131}_{0.835}$	$1.55\,^{2.70}_{0.08}$	1.542
RRT*	60.0%	$1.180\,^{1.215}_{1.162}$	$1.99\,^{4.15}_{0.80}$	0.772	60.0%	$1.206\,^{1.461}_{0.887}$	$4.58\,^{6.83}_{2.77}$	0.653	80.0%	$1.007\,^{1.144}_{0.891}$	$4.77\,^{7.76}_{0.55}$	1.762
DDPG:CAR	100.0%	$1.196\,^{1.203}_{1.182}$	$7.81\,^{8.36}_{7.31}$	1.745	100.0%	$1.057\,^{1.063}_{1.049}$	$7.68\,^{7.96}_{7.44}$	2.414	100.0%	$1.046\,^{1.119}_{1.019}$	$5.74\,^{6.09}_{4.46}$	1.763
DDPG:COR	100.0%	$1.154\,^{1.164}_{1.145}$	$6.78\,^{7.05}_{6.57}$	2.065	100.0%	$1.067\,^{1.077}_{1.060}$	$6.08\,^{6.15}_{5.95}$	2.005	100.0%	$0.965\,^{0.972}_{0.958}$	$4.70\,^{5.35}_{4.21}$	2.353
APE2:BC	100.0%	$1.144\,^{1.180}_{1.131}$	$8.46\,^{8.69}_{8.14}$	2.388	100.0%	$1.071\,^{1.077}_{1.063}$	$8.02\,^{8.66}_{7.69}$	2.430	100.0%	$1.040\,^{1.076}_{1.005}$	$6.85\,^{7.34}_{6.33}$	2.023
APE2:DC-ED2	**100.0%**	$\mathbf{1.103}\,^{1.108}_{1.096}$	$\mathbf{9.80}\,^{10.0}_{9.55}$	**3.000**	**100.0%**	$\mathbf{0.978}\,^{0.994}_{0.951}$	$\mathbf{8.73}\,^{9.05}_{8.57}$	**3.000**	**100.0%**	$\mathbf{0.942}\,^{0.949}_{0.936}$	$\mathbf{7.04}\,^{7.42}_{6.35}$	**3.000**

Annotation: The data of trajectory length as well as minimum distance between the manipulator and obstacles are in the format of $\text{mean}^{\text{max}}_{\text{min}}$.

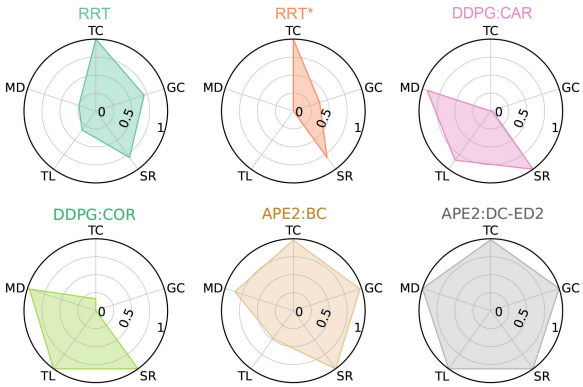


Fig. 12. Performance evaluation of different motion planners. TC: training cost. GC: generation cost. SR: success rate. TL: trajectory length. MD: MD between the manipulator and obstacle during motion.

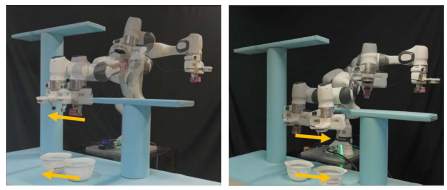


Fig. 13. Examples of trajectory replanning for changed goal poses. The robot initially executes a preplanned trajectory to transport a grasped object above a static basket. Subsequently, the basket moves in the direction indicated by the yellow arrow. URPlanner replans the subsequent motions, enabling the robot to track the moving basket.

followed by 4.293 for APE2:BC, 3.136 for DDPG:COR, 3.081 for RRT, 2.802 for DDPG:CAR, and 1.986 for RRT*.

Without loss of generality, the performances of the six planners are further evaluated by comparing the quality of the trajectories planned for several other goal poses within the designated target area. The distance between two consecutive points is 10 cm. Similar to the experiments presented in Table VII, five trajectories are generated by each planner for each goal pose, with only successful trials included in the calculations of TL and MD. Table VIII presents the SR of trajectory generation, TL, and MD for each planner. It can be observed that RRT and RRT* remain highly unstable, with significant variability in the quality of the generated trajectories across the five trials. In contrast, the other four DRL-based planners exhibit good stability and satisfactory trajectory quality, with the trajectories almost perfectly overlapping. The trajectory generation performance is scored based on these three aspects using the calculation method introduced in Fig. 12. The experimental results demonstrate that the APE2:DC-ED2 model of our URPlanner consistently outperforms the other planners in the three listed criteria. It is worth noting that the trajectory generation policies of DRL-based planners are highly similar for goal poses in close proximity to each other. Based on the experimental results in this section, it can be concluded that the APE2:DC-ED2 model of URPlanner achieves the best overall performance among the six planners.

2) Trajectory Replanning: As discussed in Sections VII-B and VII-C, the proposed platform-agnostic method, URPlanner, enables all necessary information for trajectory generation to be computed analytically. Due to this advantage, URPlanner can preplan the entire trajectory (represented as joint configuration sequences) efficiently in the parameterized space before execution on a simulated or real-world platform. Moreover, it can replan and update all untracked configuration sequences in the parameterized space concurrently with trajectory execution. As indicated in Table VII, the planning can be implemented within a few milliseconds. In Fig. 13, we present two examples of trajectory replanning to accommodate new goal positions. Initially, the Franka robot executes a trajectory preplanned by URPlanner to transport a grasped object above a static basket. Subsequently, in the two examples, the basket moves in the direction indicated by the yellow arrow. URPlanner replans the subsequent motions in the parameterized space, enabling the robot to track the basket's movement and successfully drop the object into it. Notably, even as the goal pose eventually exceeds the trained target area, URPlanner demonstrates robustness to these untrained environmental states.

VIII. DISCUSSION

A. Universality of URPlanner

In this section, we demonstrate the universality and effectiveness of URPlanner across various task scenarios, as illustrated in Fig. 15. These scenarios involve different manipulators and environments. For each scenario, we compare the training performance of DDPG, APE2, and APE2:DC-ED2 algorithms. The training SRs are summarized in Table IX, and the variation of episode rewards during training is depicted

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
TRAINING SUCCESS RATE (%) PER 250 EPISODES OF ALGORITHMS IN DIFFERENT SCENARIOS

	Scenario 1				Scenario 2				Scenario 3			
	1-250	251-500	501-750	751-1000	1-250	251-500	501-750	751-1000	1-250	251-500	501-750	751-1000
DDPG	$0.0\,^{0.0}_{0.0}$	$0.0\,^{0.0}_{0.0}$	$54.4\,^{62.0}_{37.2}$	$98.0\,^{99.6}_{94.4}$	$0.0\,^{0.0}_{0.0}$	$54.3\,^{81.6}_{14.8}$	$98.8\,^{99.6}_{97.2}$	$98.6\,^{99.2}_{97.2}$	$0.0\,^{0.0}_{0.0}$	$4.5\,^{18.0}_{0.0}$	$53.4\,^{95.6}_{27.6}$	$95.9\,^{98.0}_{93.6}$
APE2	$31.2\,^{58.0}_{4.0}$	$99.4\,^{100}_{98.4}$	$99.8\,^{100}_{99.6}$	$100\,^{100}_{100}$	$72.4\,^{78.8}_{67.2}$	$100\,^{100}_{100}$	$99.9\,^{100}_{99.6}$	$99.8\,^{100}_{99.6}$	$29.9\,^{38.4}_{11.6}$	$100\,^{100}_{100}$	$98.8\,^{99.6}_{96.4}$	$99.7\,^{100}_{99.2}$
APE2:DC-ED2	$69.3\,^{79.2}_{63.6}$	$100\,^{100}_{100}$	$99.4\,^{99.6}_{98.8}$	$99.8\,^{100}_{99.2}$	$78.3\,^{81.6}_{74.8}$	$99.9\,^{100}_{99.6}$	$98.8\,^{99.6}_{98.4}$	$99.9\,^{100}_{99.6}$	$66.5\,^{69.2}_{65.2}$	$100\,^{100}_{100}$	$99.1\,^{100}_{97.2}$	$99.9\,^{100}_{99.6}$

Annotation: The data are in the format of $\text{mean}^{\text{max}}_{\text{min}}$. In each scenario, all the algorithms achieve a 100% testing success rate based on 50 trials.

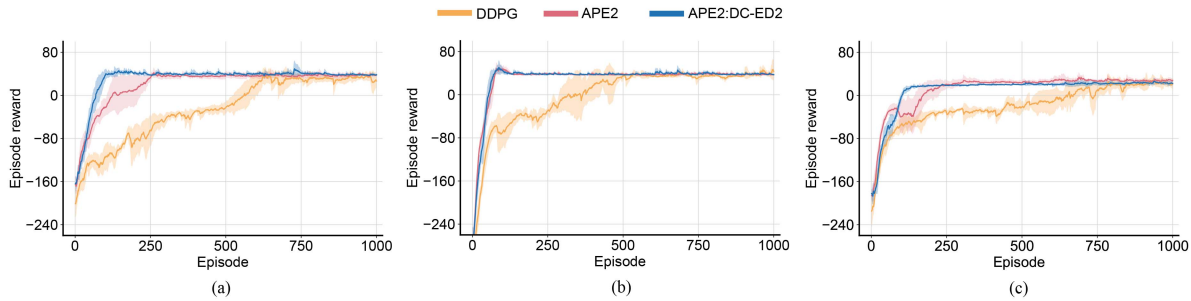


Fig. 14. Variation of episode rewards during training for DDPG, APE, and APE2:DC-ED2 algorithms in different scenarios. The shaded region represents the standard deviation of the average evaluation over four random seeds. (a) Scenario 1: training a KUKA robot. (b) Scenario 2: training a Franka robot in a simple environment. (c) Scenario 3: training a Franka robot in a complex environment.

in Fig. 14. The benchmark algorithm, DDPG, exhibits much slower policy convergence compared to the proposed APE2 and APE2:DC-ED2 algorithms across all scenarios. In contrast, APE2 achieves stable policy learning within approximately 250 episodes in more complex environments, such as Scenarios 1 and 3. Scenario 2, being significantly simpler, allows the APE2 algorithms to learn a stable policy in only about 80 episodes. A comprehensive comparison of training SRs during the early stages (1st to 500th episodes) reveals that the proposed APE2 algorithm demonstrates substantial improvements over DDPG.

Despite the powerful learning ability APE exhibits in handling environments with different levels of complexity, its performance can be further enhanced by the proposed ED2 approach. Experimental results indicate that APE2:DC-ED2 achieves the best training performance across all scenarios, particularly during the first 250 episodes, where it learns a stable policy within approximately 70 episodes for Scenarios 1 and 3. Notably, the improvement in training SR brought by APE2:DC-ED2 in Scenario 2 is limited, as APE2 already performs exceptionally well in this simple environment. In practice, expert data are primarily required to address highly complex scenarios.

These findings demonstrate that URPlanner is a universal paradigm capable of supporting different manipulators and environments. The proposed techniques, including collision avoidance, policy exploration and evaluation, and expert data diffusion and utilization, are effective across diverse scenarios. Furthermore, Fig. 15 highlights the platform-agnostic nature of URPlanner, as policies trained in the parameterized space can be directly applied to both simulated and real-world platforms without fine-tuning.

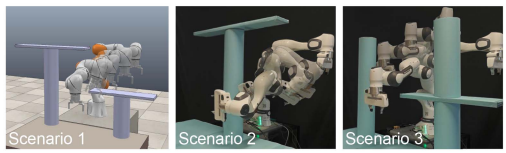


Fig. 15. Universality of URPlanner across different manipulators and environments. Scenario 1: training a KUKA robot. Scenario 2: training a Franka robot in a simple environment. Scenario 3: training a Franka robot in a complex environment.

B. Generalization Across Different Scenarios

The generalization ability is a common challenge in DRL, as optimal policies often vary across different environments. Typically, if the new environment closely resembles the training environment, such as the experiments in Table V, the model may generalize reasonably well. However, in cases where the new environment differs significantly, such as featuring entirely new obstacles, manipulators, or target areas, the model may struggle to perform effectively without retraining. Studies including [7], [9], and [36] emphasize that retraining is often required when applying a learned policy to new task scenarios.

In this work, we abstract obstacles as one or multiple bounding boxes in the proposed parameterized space. This abstraction ensures that the trained motion planning policy is universally applicable to all objects that can be enclosed by the bounding box used during training. While training a single DRL policy to handle all possible task scenarios is inherently challenging, the introduction of a policy library enables our method to generalize effectively across various scenarios.

As illustrated by the simple example in Fig. 16, the workspace on a desk is divided into three regions: 1, 2, and 3. Three subpolicies, each tailored to a respective subworkspace, are trained. To train these policies, each time, one region is defined as a subworkspace, with the remaining two regions treated as obstacle spaces with a certain height. These subpolicies, represented by network parameters, are stored in the policy library

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

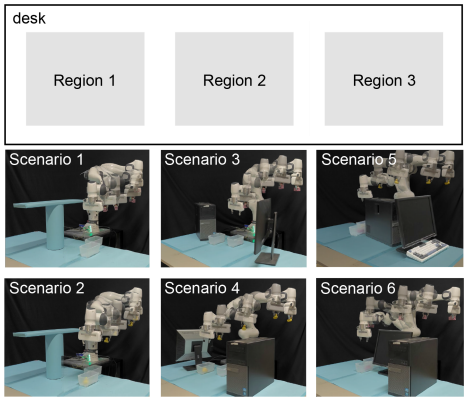


Fig. 16. Examples of enhancing generalization by developing a policy library. Six new scenarios involving random obstacle and target layouts are given, and the agent can retrieve an appropriate subpolicy from the library to generate collision-free motions.

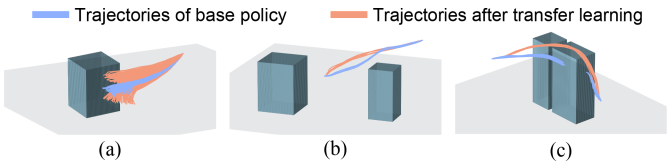


Fig. 17. Examples of leveraging transfer learning for efficient policy adaptation to new environments. (a) Policy adaptation for new target areas. (b) Policy adjustment for smaller obstacles. (c) Policy adaptation for larger obstacles.

along with the spatial occupancy data of the target and obstacle regions.

The combination of these three subpolicies enables coverage of a broad range of obstacle and target layouts, provided that the target is not in the same region as the obstacles. As illustrated in Fig. 16, six scenarios with random goal positions and/or obstacles are established. In Scenarios 1 and 2, the two boxes are located in Region 1, where the obstacles in Region 2 have minimal influence on the robot's motion to the goals. In Scenarios 3 and 4, the boxes are located in Region 2, while the two obstacles are positioned randomly in Regions 1 and 3, respectively. In Scenarios 5 and 6, the robot needs to navigate across two consecutive obstacle regions to reach the goals in Region 3. These examples demonstrate that the agent can flexibly retrieve an appropriate subpolicy from the library to generate collision-free motions, even when faced with new scenarios involving random obstacle and target configurations.

If no feasible subpolicy exists in the library, i.e., an obstacle and target configuration not covered by the existing policies, a new subpolicy is automatically trained and added to the library. The cost effectiveness of our URPlanner proposed in this work enables this training process to be completed within a few minutes, compared to the hours required by conventional DRL-based planners. As the policy library expands with additional subpolicies, the generalization capability of our approach continuously improves.

In addition, the TC for a new subpolicy can be further reduced by leveraging transfer learning [7]. For example, Fig. 17(a) illustrates the policy adaptation for new target areas where the original target area is raised or lowered by 10 cm. The new policies are trained within about 150 s by transferring the base policy. In Fig. 17(c), the obstacles are stretched, causing the base policy unable to generate collision-free trajectories. Within about 75 s of transfer learning, the base policy is adjusted to stably generate collision-free trajectories for reaching random targets in Region 3.

While the policy library effectively improves generalization, it may compromise the quality of the generated trajectories. This is particularly evident when the bounding box used for training the subpolicy is significantly larger than the obstacles in new scenarios, leading to unnecessary safety margins and inefficiencies in reaching the target. In such situations, transfer learning can be employed to refine the trajectory generated by the subpolicy. Fig. 17(b) illustrates an example where transfer learning refines the base policy for reaching random targets in Region 2, reducing the average TL by 4.6 cm and increasing the MD to obstacles by 4.3 cm.

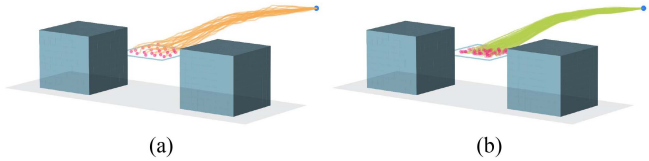


Fig. 18. Example of demonstration provision and novel trajectory generation by ED2. The designated target area is uniformly divided into 25 subareas, with demonstrations provided for each subarea. (a) Twenty-five provided demonstrations. (b) One hundred novel trajectories generated by ED2.

TABLE X
TRAINING PERFORMANCE OF THE APE2:DC-ED2 ALGORITHM WITH VARYING NUMBERS OF EXPERT DEMONSTRATIONS

Algorithm	Number of demonstrations	Success Rate Per 250 Episodes (%)			
		1-250	251-500	501-750	751-1000
APE2	–	$7.8\,^{25.2}_{0.4}$	$86.9\,^{98.8}_{75.2}$	$100\,^{100}_{100}$	$99.5\,^{100}_{99.2}$
APE2 with DC-ED2	1	$43.2\,^{48.0}_{36.8}$	$99.9\,^{100}_{99.6}$	$100\,^{100}_{100}$	$99.7\,^{100}_{99.2}$
	5	$45.6\,^{48.8}_{43.2}$	$100\,^{100}_{100}$	$100\,^{100}_{100}$	$99.7\,^{100}_{99.6}$
	10	$45.8\,^{53.6}_{36.4}$	$100\,^{100}_{100}$	$99.9\,^{100}_{99.6}$	$99.7\,^{100}_{99.2}$
	20	$47.1\,^{50.4}_{41.6}$	$99.8\,^{100}_{99.6}$	$99.9\,^{100}_{99.6}$	$100\,^{100}_{100}$

Annotation: The data are in the format of $\text{mean}^{\max}_{\min}$ over four random seeds. All the algorithms achieve a 100% testing success rate based on 50 trials.

C. Strategy for Providing Demonstrations to ED2

As discussed in Section VII-C, the trajectory generation policies of DRL-based planners are highly similar for goal poses in close proximity to each other. Thus, a designated target area can be uniformly divided into several subareas, with demonstrations provided for each subarea. For example, in Fig. 18(a), one demonstration is provided for each of the 25 subareas. Fig. 18(b) shows 100 novel trajectories generated by the proposed ED2 model, trained using the 25 demonstrations shown in Fig. 18(a). These results demonstrate that the ED2 model is capable of generating diverse novel trajectories from a limited number of expert demonstrations.

As shown in Table X, by setting the number of demonstrations provided to each subarea as 1, 5, 10, and 20, respectively, four ED2 models are trained accordingly. Subsequently, four expert memories are generated based on these trained ED2

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

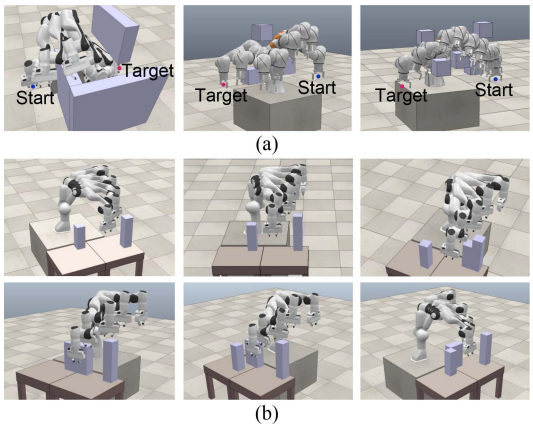


Fig. 19. Extensions of URPlanner. (a) Approximating the robot's link with multiple line segments eliminates the need to expand obstacle representations and enhances URPlanner's planning ability in narrow spaces. (b) URPlanner can be extended to handle environments with randomized obstacle configurations by incorporating obstacle information as a 3-D occupancy grid matrix in the state representation.

models. These expert memories are used by the DC mechanism, referred to as DC-ED2, to evaluate their impact on facilitating the training of the APE2 algorithm. The results demonstrate that the APE2:DC-ED2 algorithm with ED2 trained on a single demonstration per subarea achieves an SR of 43.2% in the first 250 training episodes. This performance is only marginally lower than the SRs achieved by the APE2:DC-ED2 using ED2 models trained on 5, 10, and 20 demonstrations per subarea, respectively. These results demonstrate that ED2, trained with a limited number of demonstrations, is capable of generating high-quality data to accelerate the policy convergence of DRL.

D. Extension of URPlanner

As discussed in Section III-A, URPlanner adopts a clearance-based motion planning strategy by expanding the bounding boxes that represent obstacles. This expansion ensures that the robot maintains a safe distance from potential collisions. However, this conservative approach may limit the planner's ability to navigate through environments with narrow passages or tightly clustered obstacles.

URPlanner can be easily extended to overcome this limitation. Specifically, the surface of each cylinder that represents the robot's link can be approximated by multiple line segments, similar to the method in [29]. The computation of collision detection and the UOAR remains unchanged. This approach eliminates the need to expand obstacle representations and avoids reducing the cylindrical model to a single central line segment. Although this extension slightly increases the reward computation time, it significantly improves URPlanner's planning ability in constrained environments. As demonstrated in Fig. 19(a), the extended URPlanner effectively guides both KUKA and Franka manipulators to execute smooth collision-free curved trajectories, successfully navigating narrow spaces between ground-level and elevated obstacles.

Moreover, URPlanner can be extended to handle environments with random obstacle layouts. In this work, we assume static obstacle layouts and only the target positions vary randomly across episodes. In this context, the state s_t defined in (2) is sufficient. This state setup is consistent with many prior works in DRL-based motion planning, such as [7], [8], and [9].

To handle environments with random obstacle layouts, obstacle information must be incorporated into the state variable. A straightforward yet limited solution adopted in prior works [26], [30], [37] is to include the positions of the obstacles. This method works when the number and shape of obstacles are constant, but it becomes impractical in highly variable settings.

To enable scalability, we propose using a 3-D occupancy grid with a fixed resolution to represent the spatial configuration of obstacles. This representation allows the state variable to remain fixed in dimension while encoding arbitrary obstacle shapes and distributions. As shown in Fig. 19(b), by incorporating the occupancy grid into s_t, URPlanner is capable of handling environments with randomized obstacle configurations. The robot successfully learns to navigate around obstacles whose number, size, and shape change across episodes. In future work, we plan to further optimize this approach.

IX. Conclusion

This article presents a universal paradigm for collision-free robotic motion planning named URPlanner, which is platform agnostic and highly cost-effective in both training and deployment. This efficiency is attributed to integrating the proposed parameterized task space, MD-independent UOAR, powerful APE2 algorithm, and ED2 model with the DC mechanism. For a given environment, URPlanner can be trained within a few minutes. The experimental results have also shown that the trained model can plan IK-free, collision-free, and near-optimal motion within a few milliseconds. In contrast, other methods may take up to several seconds or longer. Moreover, our URPlanner demonstrates good robustness and adaptability across various task scenes, platforms, and manipulators, and generalizes well to untrained environmental states.

One limitation of our work is that the approximation of obstacles and manipulator links may lead to slight underutilization of free space, as some collision-free regions are conservatively treated as unsafe. However, this simplification provides significant gains in computational efficiency. In practice, these regions can serve as parts of the safety margins, thereby improving clearance during motion planning. For future work, we think it would be interesting to explore the application of our proposed URPlanner in dynamic environments with moving obstacles. Particularly, a human partner may also be modeled as a moving obstacle so that the approach can also be used in human–robot collaboration scenarios.

References

[1] J. Lv, Y. Wang, S. Wang, X. Bai, R. Wang, and M. Tan, "A collision-free planning and control framework for a biomimetic underwater vehicle in dynamic environments," *IEEE/ASME Trans. Mechatron.*, vol. 28, no. 3, pp. 1415–1424, Jun. 2023.

[2] M. Safeea, R. Bearee, and P. Neto, "A modified DLS scheme with controlled cyclic solution for inverse kinematics in redundant robots," *IEEE Trans. Ind. Informat.*, vol. 17, no. 12, pp. 8014–8023, Dec. 2021.

[3] A. H. Qureshi, Y. Miao, A. Simeonov, and M. C. Yip, "Motion planning networks: Bridging the gap between learning-based and classical motion planners," *IEEE Trans. Robot.*, vol. 37, no. 1, pp. 48–66, Feb. 2021.

[4] S. Ruan, K. L. Poblete, H. Wu, Q. Ma, and G. S. Chirikjian, "Efficient path planning in narrow passages for robots with ellipsoidal components," *IEEE Trans. Robot.*, vol. 39, no. 1, pp. 110–127, Feb. 2023.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.

[5] X. Li, H. Liu, and M. Dong, "A general framework of motion planning for redundant robot manipulator based on deep reinforcement learning," *IEEE Trans. Ind. Informat.*, vol. 18, no. 8, pp. 5253–5263, Aug. 2022.
[6] A. Faust et al., "PRM-RL: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018, pp. 5113–5120.
[7] H. Liu, F. Ying, R. Jiang, Y. Shan, and B. Shen, "Obstacle-avoidable robotic motion planning framework based on deep reinforcement learning," *IEEE/ASME Trans. Mechatron.*, vol. 29, no. 6, pp. 4377–4388, Dec. 2024.
[8] F. Ying, H. Liu, R. Jiang, and M. Dong, "Extensively explored and evaluated actor-critic with expert-guided policy learning and fuzzy feedback reward for robotic trajectory generation," *IEEE Trans. Ind. Informat.*, vol. 18, no. 11, pp. 7749–7760, Nov. 2022.
[9] F. Ying, H. Liu, R. Jiang, and X. Yin, "Trajectory generation for multiprocess robotic tasks based on nested dual-memory deep deterministic policy gradient," *IEEE/ASME Trans. Mechatron.*, vol. 27, no. 6, pp. 4643–4653, Dec. 2022.
[10] S. Safaoui, A. P. Vinod, A. Chakrabarty, R. Quirynen, N. Yoshikawa, and S. D. Cairano, "Safe multiagent motion planning under uncertainty for drones using filtered reinforcement learning," *IEEE Trans. Robot.*, vol. 40, pp. 2529–2542, 2024.
[11] R. Parák, J. Kůdela, R. Matoušek, and M. Juříček, "Deep-reinforcement-learning-based motion planning for a wide range of robotic structures," *Computation*, vol. 12, no. 6, 2024, Art. no. 116.
[12] Z. Hu, Y. Zheng, and J. Pan, "Grasping living objects with adversarial behaviors using inverse reinforcement learning," *IEEE Trans. Robot.*, vol. 39, no. 2, pp. 1151–1163, Apr. 2023.
[13] A. Zeng, S. Song, S. Welker, J. Lee, A. Rodriguez, and T. Funkhouser, "Learning synergies between pushing and grasping with self-supervised deep reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 4238–4245.
[14] Y. Deng et al., "Deep reinforcement learning for robotic pushing and picking in cluttered environment," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 619–626.
[15] L. Berscheid, P. Meißner, and T. Kröger, "Robot learning of shifting objects for grasping in cluttered environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 612–618.
[16] Z. Hou, Z. Li, C. Hsu, K. Zhang, and J. Xu, "Fuzzy logic-driven variable time-scale prediction-based reinforcement learning for robotic multiple peg-in-hole assembly," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 1, pp. 218–229, Jan. 2022.
[17] Z. Hou, W. Yang, R. Chen, P. Feng, and J. Xu, "A hierarchical compliance-based contextual policy search for robotic manipulation tasks with multiple objectives," *IEEE Trans. Ind. Informat.*, vol. 19, no. 4, pp. 5444–5455, Apr. 2023.
[18] H. Xu, Y. Luo, S. Wang, T. Darrell, and R. Calandra, "Towards learning to play piano with dexterous hands and touch," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 10410–10416.
[19] D. Bianchi, M. G. Antonelli, C. Laschi, A. M. Sabatini, and E. Falotico, "SofToss: Learning to throw objects with a soft robot," *IEEE Robot. Autom. Mag.*, vol. 31, no. 4, pp. 113–123, Dec. 2024.
[20] L. Pecyna, S. Dong, and S. Luo, "Visual-tactile multimodality for following deformable linear objects using reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 3987–3994.
[21] Y. Chen, F. Ying, X. Li, and H. Liu, "Deep reinforcement learning in maximum entropy framework with automatic adjustment of mixed temperature parameters for path planning," in *Proc. IEEE 7th Int. Conf. Robot. Control Autom.*, 2023, pp. 78–82.
[22] Z. Sui, Z. Pu, J. Yi, and S. Wu, "Formation control with collision avoidance through deep reinforcement learning using model-guided demonstration," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 6, pp. 2358–2372, Jun. 2021.
[23] W. Liu et al., "Ensemble bootstrapped deep deterministic policy gradient for vision-based robotic grasping," *IEEE Access*, vol. 9, pp. 19916–19925, 2021.
[24] J. Wu, R. Wang, R. Li, H. Zhang, and X. Hu, "Multi-critic DDPG method and double experience replay," in *Proc. IEEE Int. Conf. Syst., Man, Cybern.*, 2018, pp. 165–171.
[25] Y. Ma, D. Xu, and F. Qin, "Efficient insertion control for precision assembly based on demonstration learning and reinforcement learning," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4492–4502, Jul. 2021.
[26] P. Chen, J. Pei, W. Lu, and M. Li, "A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance," *Neurocomputing*, vol. 497, pp. 64–75, 2022.
[27] A. Xie, Y. Lee, P. Abbeel, and S. James, "Language-conditioned path planning," in *Proc. 7th Conf. Robot Learn.*, 2023, pp. 3384–3396.
[28] Z. Hou, J. Fei, Y. Deng, and J. Xu, "Data-efficient hierarchical reinforcement learning for robotic assembly control applications," *IEEE Trans. Ind. Electron.*, vol. 68, no. 11, pp. 11565–11575, Nov. 2021.
[29] Z. Mu, W. Xu, X. Gao, L. Xue, and C. Li, "Obstacles modeling and collision detection of space robots for performing on-orbit services," in *Proc. 4th IEEE Int. Conf. Inf. Sci. Technol.*, 2014, pp. 461–466.
[30] S. Yang and Q. Wang, "Robotic arm motion planning with autonomous obstacle avoidance based on deep reinforcement learning," in *Proc. IEEE 41st Chin. Control Conf.*, 2022, pp. 3692–3697.
[31] Y.-L. Kim, K.-H. Ahn, and J.-B. Song, "Reinforcement learning based on movement primitives for contact tasks," *Robot. Comput.- Integr. Manuf.*, vol. 62, 2020, Art. no. 101863.
[32] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 6840–6851.
[33] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 9902–9915.
[34] H. Liu, J. Tan, Y. Chen, and M. H. Ang, "Learning complicated manipulation skills via deterministic policy with limited demonstrations," in *Proc. 21st Int. Conf. Adv. Robot.*, 2023, pp. 499–505.
[35] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, "Efficient online reinforcement learning with offline data," in *Proc. 40th Int. Conf. Mach. Learn.*, 2023, pp. 1577–1594.
[36] J. Xu, Z. Hou, W. Wang, B. Xu, K. Zhang, and K. Chen, "Feedback deep deterministic policy gradient with fuzzy reward for robotic multiple peg-in-hole assembly tasks," *IEEE Trans. Ind. Informat.*, vol. 15, no. 3, pp. 1658–1667, Mar. 2019.
[37] Z. Zhang, J. Hong, A. M. S. Enayati, and H. Najjaran, "Using implicit behavior cloning and dynamic movement primitive to facilitate reinforcement learning for robot motion planning," *IEEE Trans. Robot.*, vol. 40, pp. 4733–4749, 2024.



Fengkang Ying (Graduate Student Member, IEEE) received the M.E. degree in electrical engineering from Donghua University, Shanghai, China, in 2022. He is currently working toward the Ph.D. degree with the Integrative Sciences and Engineering Programme (ISEP), NUS Graduate School, National University of Singapore (NUS), Singapore.

Since August 2023, he has been with the ISEP and NUS Advanced Robotics Centre, NUS. Since January 2025, he has been a Visiting Student with the Mens, Manus, and Machina Interdisciplinary Research Group, Singapore MIT Alliance for Research and Technology Centre, Singapore. His current research interests include robotics, task and motion planning, and reinforcement learning.



Hanwen Zhang received the B.E. degree in material science and engineering with a minor in robotics from the University of Science and Technology Beijing, Beijing, China, in 2022, and the M.Sc. degree in robotics from the National University of Singapore, Singapore, in 2025.

His research interests include intelligent robotic systems, machine learning, and motion planning.



Haozhe Wang (Graduate Student Member, IEEE) received the B.E. (Hons.) degree in electrical engineering (distinction) with a specialization in Internet of Things in 2021 from the National University of Singapore (NUS), Singapore, where he is currently working toward the Ph.D. degree with the Integrative Sciences and Engineering Programme (ISEP), NUS Graduate School.

Since August 2021, he has been with the ISEP and NUS Advanced Robotics Centre. Since June 2024, he has been a Visiting Student with the Mens, Manus, and Machina Interdisciplinary Research Group, Singapore MIT Alliance for Research and Technology Centre, Singapore. His current research interests include robotic grasping and manipulation, computer vision, and learning from demonstrations.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.



Huishi Huang (Graduate Student Member, IEEE) received the M.E. degree in physics and engineering in medicine from University College London, London, U.K., in 2022. She is currently working toward the Ph.D. degree in mechanical engineering with the National University of Singapore (NUS), Singapore.

Since August 2023, she has been with the Department of Mechanical Engineering, NUS, and the Institute of High Performance Computing, Agency for Science, Technology and Research, Singapore. Since January 2025, she has been a Visiting Student with the Mens, Manus, and Machina Interdisciplinary Research Group, Singapore MIT Alliance for Research and Technology Centre, Singapore. Her research interests include bioinspired soft manipulators, machine learning, and mechatronics.



Marcelo H. Ang Jr. (Senior Member, IEEE) received the B.Sc. degree (cum laude) in mechanical engineering and industrial management engineering from the De La Salle University, Manila, Philippines, in 1981, the M.Sc. degree in mechanical engineering from the University of Hawaii at Manoa, Honolulu, HI, USA, in 1985, and the M.Sc. and Ph.D. degrees in electrical engineering from the University of Rochester, Rochester, NY, USA, in 1986 and 1988, respectively.

His work experience includes heading the Technical Training Division of Intel's Assembly and Test Facility in the Philippines, research positions with the East-West Center, Honolulu, and the Massachusetts Institute of Technology, Cambridge, MA, USA, and a Faculty Position as an Assistant Professor of Electrical Engineering with the University of Rochester. In 1989, he joined the Department of Mechanical Engineering, National University of Singapore, Singapore, where he is currently a Professor. He is also the Director of the Advanced Robotics Centre, National University of Singapore. He teaches both graduate and undergraduate levels in the following areas: robotics, creativity and innovation, and engineering mathematics. He is also active in consulting work in robotics and intelligent systems. In addition to academic and research activities, he is actively involved in the Singapore Robotic Games as its founding Chairman and the World Robot Olympiad as a Member of the Advisory Council. His research interests include robotics, mechatronics, and the application of intelligent systems methodologies.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:41:54 UTC from IEEE Xplore. Restrictions apply.